TRABAJO PRÁCTICO: DESARROLLO DE ALGORITMOS

Diagrama de flujo • Pseudocódigo • Prueba de escritorio • Codificación en C • Conclusiones

1. Datos informativos

Asignatura:	Teoría de la Programación
Docente:	Jose Oswaldo Guaman
Estudiante(s):	Josué Alejandro González Salas
Curso / Paralelo:	1 A
Fecha de entrega:	29/10/2026
Tema:	Desarrollo de algoritmos: representación gráfica, pseudocódigo, prueba de escritorio y codificación en C

2. Objetivo

Desarrollar los algoritmos propuestos por el docente aplicando las distintas formas de representación y construcción de soluciones: diagrama de flujo, pseudocódigo, prueba de escritorio y su implementación en el lenguaje de programación C.

3. Instrucciones generales

1. El trabajo se realizará de forma **individual** (o en grupos de [__] personas).
2. Se deben desarrollar **todos los algoritmos enviados** por el docente.
3. Por **cada algoritmo** se debe presentar la estructura indicada en el punto 4, respetando el orden establecido.
4. Los diagramas de flujo pueden elaborarse en herramientas como **draw.io, Flowgorithm o Lucidchart**, o a mano legible (fotografiados).
5. La codificación debe realizarse en **lenguaje C**, compilada y probada. Se debe incluir una **captura de pantalla de la salida** del programa con datos de ejemplo.
6. El documento se entregará en formato **PDF** a través de [medio de entrega].

7. La copia total o parcial de trabajos será sancionada según el reglamento institucional.
8. El docente podrá solicitar una **sustentación oral** del trabajo para verificar la autoría de las soluciones presentadas.

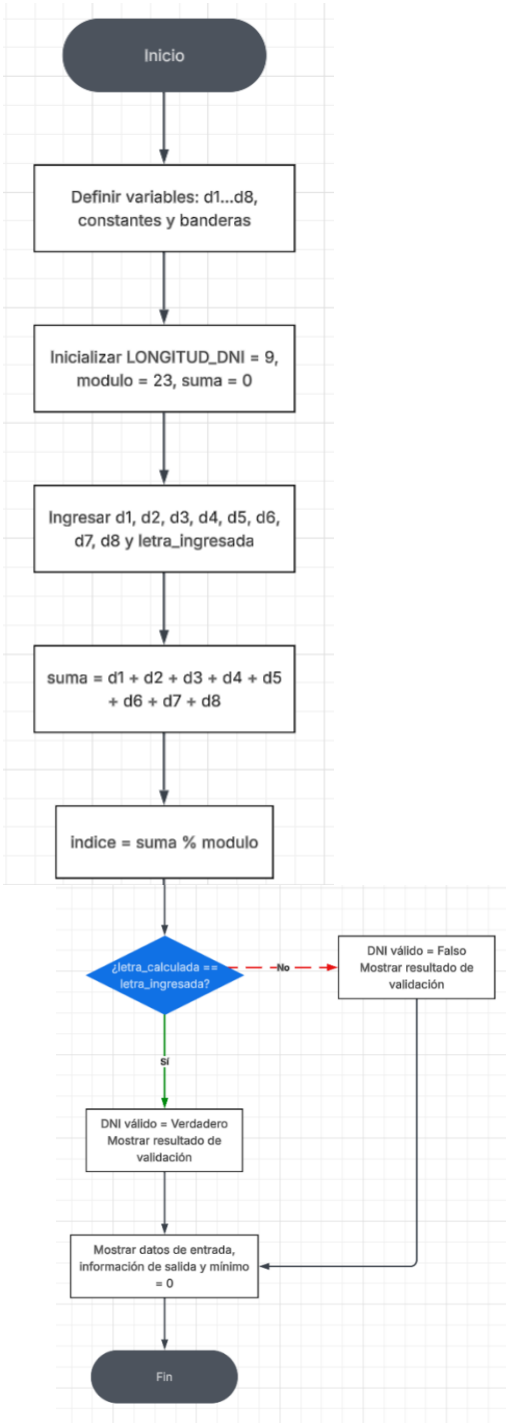
> 🚫 **PROHIBIDO EL USO DE INTELIGENCIA ARTIFICIAL:** queda estrictamente prohibido utilizar herramientas de IA (ChatGPT, Gemini, Copilot, DeepSeek, Claude, entre otras) para generar o resolver los diagramas de flujo, pseudocódigo, pruebas de escritorio, código en C o conclusiones. El trabajo debe ser de **elaboración 100% propia**. El incumplimiento de esta norma se calificará con **0 (cero)** y se aplicará el reglamento institucional correspondiente.

4. Desarrollo de los 20 ejercicios: (pseudocódigo, diagrama de flujo y código)

Ejercicio 1:
En una tienda de ropa, diseñe un algoritmo que valide un DNI (cadena de 8 dígitos + 1 letra). Use constantes simbólicas LONGITUD_DNI = 9, MODULO = 23. Use una variable entera suma inicializada con literal 0. Calcule la suma de los 8 dígitos con expresiones aritméticas. Calcule el índice de letra con (suma % MODULO). Evalúe la expresión relacional (letra_calculada == letra_ingresada). Asigne el resultado a una variable booleana dni_valido. Muestre por pantalla la validación. Además, calcule el valor mínimo de todas las variables utilizadas. Identifique explícitamente los datos de entrada y la información de salida.

Analisis de datos:
Datos de entrada: Los 8 dígitos del DNI: Ingresados de forma individual (d1 a d8)
La letra ingresada: El carácter correspondiente al DNI que proporciona el usuario (letra_ingresada).
suma = d1 + d2 + d3 + d4 + d5 + d6 + d7 + d8
Evaluación relacional: Comparación entre la letra calculada y la letra ingresada:
dni_valido = (textletra_calculada == letra_ingresada)
Valor mínimo: Determinación del valor mínimo que pueden tomar las variables numéricas del algoritmo, el cual es 0.
Datos de salida: Resultado de la validación: El estado lógico almacenado en la variable booleana dni_valido.
LONGITUD_DNI (Constante Entera):
MODULO (Constante Entera):
d1, d2, d3, d4, d5, d6, d7, d8 (como entero):
suma (como entero):
indice (como entero):
letra_ingresada (como carácter):
letra_calculada (como carácter):
dni_valido (como booleano / lógico):

AlgoritmoDiagrama de flujo:
Figura [1]



A continuación, realizaremos el pseudocódigo:
Inicio
Definir d1, d2, d3, d4, d5, d6, d7, d8, suma, resto Como entero;
Definir letra_ingresada, letra_calculada Como carácter;

Escribir "Ingrese los 8 digitos del DNI uno por uno";
Leer d1;
Leer d2;
Leer d3;
Leer d4;
Leer d5;
Leer d6;
Leer d7;
Leer d8;
Escribir "Ingrese la letra del DNI";
Leer letra_ingresada;

suma = d1 + d2 + d3 + d4 + d5 + d6 + d7 + d8;
resto = suma % 23;

Si resto == 0 entonces letra_calculada = "T";
Si resto == 1 entonces letra_calculada = "R";
Si resto == 2 entonces letra_calculada = "W";
Si resto == 3 entonces letra_calculada = "A";
Si resto == 4 entonces letra_calculada = "G";
Si resto == 5 entonces letra_calculada = "M";
Si resto == 6 entonces letra_calculada = "Y";
Si resto == 7 entonces letra_calculada = "F";
Si resto == 8 entonces letra_calculada = "P";
Si resto == 9 entonces letra_calculada = "D";
Si resto == 10 entonces letra_calculada = "X";
Si resto == 11 entonces letra_calculada = "B";
Si resto == 12 entonces letra_calculada = "N";
Si resto == 13 entonces letra_calculada = "J";
Si resto == 14 entonces letra_calculada = "Z";
Si resto == 15 entonces letra_calculada = "S";
Si resto == 16 entonces letra_calculada = "Q";
Si resto == 17 entonces letra_calculada = "V";
Si resto == 18 entonces letra_calculada = "H";
Si resto == 19 entonces letra_calculada = "L";
Si resto == 20 entonces letra_calculada = "C";
Si resto == 21 entonces letra_calculada = "K";
Si resto == 22 entonces letra_calculada = "E";

Si letra_calculada == letra_ingresada entonces;
Escribir "DNI valido";
Sino;
Escribir "DNI no valido";
Fin Si
Fin
Prueba de escritorio:
Caso 1 DNI valido – 12345678N

Variable	Operación o valor	Resultado
d1, d2, d3, d4, d5, d6, d7, d8	Dato ingresado	1, 2, 3, 4, 5, 6, 7, 8
suma	1 + 2 + 3 + 4 + 5 + 6 + 7 + 8	36
MODULO	Constante	23
resto	36 % 23	13
letra_ingresada	Dato ingresado	N
letra_calculada	Letra correspondiente al índice 13	N
dni_valido	letra_calculada == letra_ingresada	Verdadero (1)
salida_pantalla	Mensaje de validación	DNI valido

Caso 2: DNI no valido – 87654321A

Variable	Operación o valor	Resultado
d1, d2, d3, d4, d5, d6, d7, d8	Dato ingresado	8, 7, 6, 5, 4, 3, 2, 1
suma	8 + 7 + 6 + 5 + 4 + 3 + 2 + 1	36
resto	36 % 23	13
letra_ingresada	Dato ingresado	A
letra_calculada	Letra correspondiente al índice 13	N
dni_valido	letra_calculada == letra_ingresada	Falso (0)
salida_pantalla	Mensaje de validación	DNI no valido

Codificación en lenguaje C

Código:

Figura [2]

ejercicio1.c > main()

```c
#include <stdio.h>

int main() {

    int d1, d2, d3, d4, d5, d6, d7, d8;
    int longitud_dni, modulo, suma, indice;
    int letra_ingresada;
    int letra_calculada = 0;
    int dni_valido;

    longitud_dni = 9;
    modulo = 23;
    suma = 0;

    printf("Ingrese el primer digito del DNI: ");
    scanf("%d", &d1);
    printf("Ingrese el segundo digito del DNI: ");
    scanf("%d", &d2);
    printf("Ingrese el tercer digito del DNI: ");
    scanf("%d", &d3);
    printf("Ingrese el cuarto digito del DNI: ");
    scanf("%d", &d4);
    printf("Ingrese el quinto digito del DNI: ");
    scanf("%d", &d5);
    printf("Ingrese el sexto digito del DNI: ");
    scanf("%d", &d6);
    printf("Ingrese el septimo digito del DNI: ");
    scanf("%d", &d7);
    printf("Ingrese el octavo digito del DNI: ");
    scanf("%d", &d8);

    printf("Ingrese el valor numerico de la letra del DNI: ");
    scanf("%d", &letra_ingresada);

    suma = d1 + d2 + d3 + d4 + d5 + d6 + d7 + d8;
    indice = suma % modulo;

    dni_valido = (letra_calculada == letra_ingresada);
```

Salida del programa

Figura [3]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio1.c -o ejercicio1.exe

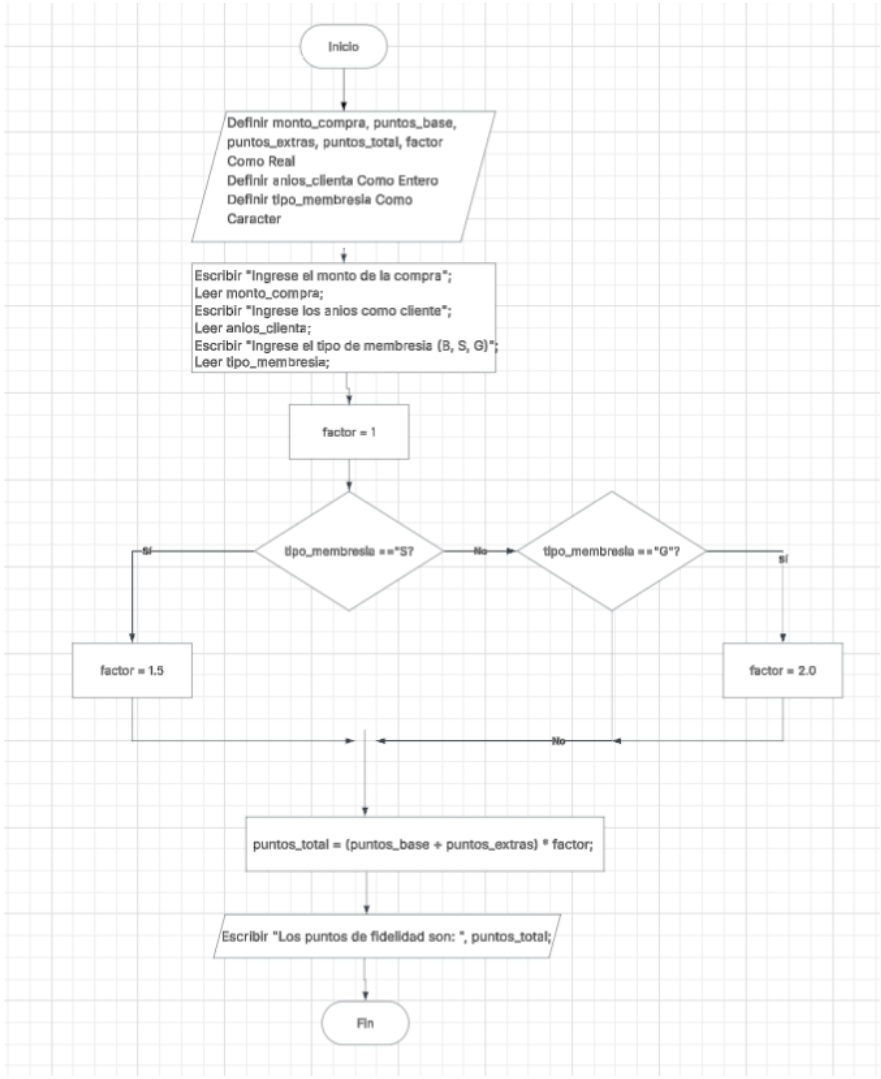
C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio1.exe
Ingrese el primer digito del DNI: 1
Ingrese el segundo digito del DNI: 5
Ingrese el tercer digito del DNI: 3
Ingrese el cuarto digito del DNI: 5
Ingrese el quinto digito del DNI: 6
Ingrese el sexto digito del DNI: 4
Ingrese el septimo digito del DNI: 7
Ingrese el octavo digito del DNI: 2
Ingrese el valor numerico de la letra del DNI: s
El DNI ingresado es VALIDO.

--- Informacion del Algoritmo ---
Datos de entrada: Los 8 digitos y el valor numerico de la letra
Informacion de salida: Variable dni_valido y mensaje en pantalla
El valor minimo de las variables utilizadas es 0
```

Ejercicio 2: En una farmacia, construya un algoritmo que calcule los puntos de fidelidad de un cliente. Lea el monto de compra (real), los años como cliente (entero) y el tipo de membresia (caracter: 'B' bronze, 'S' silver, 'G' gold). Use constantes simbolicas PUNTOS_POR_DOLAR = 1, BONO_AÑOS = 50, FACTOR_B = 1.0, FACTOR_S = 1.5, FACTOR_G = 2.0. Use expresiones logicas para seleccionar el factor. Calcule: puntos_base = monto * PUNTOS_POR_DOLAR; puntos_extra = años * BONO_AÑOS; puntos_total = (puntos_base + puntos_extra) * FACTOR. Muestre por pantalla los puntos. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de inicializar todas las variables antes de usarlas.

Analisis de datos:
Datos de entrada:
Monto de compra: Valor real proporcionado por el usuario (monto_compra).
Años como cliente: Valor entero que indica la antigüedad del cliente (annios_clienta).
Tipo de membresía: Carácter que indica la categoría ('B' para bronze, 'S' para silver, 'G' para gold).
Si es 'B' o 'b', FACTOR = FACTOR_B (1.0).
Si es 'S' o 's', FACTOR = FACTOR_S (1.5).
Si es 'G' o 'g', FACTOR = FACTOR_G (2.0).
Cálculo de puntos base:
puntos_base = monto_compra * PUNTOS_POR_DOLAR
Cálculo de puntos extra:
puntos_extras = annios_clienta * BONO_ANNIOS
Cálculo de puntos totales:
puntos_total = puntos_base + puntos_extras * FACTOR
Valor mínimo: Determinación del valor mínimo que pueden tomar las variables numéricas del algoritmo, el cual es 0.
Datos de salida:
Puntos de fidelidad: El resultado numérico almacenado en la variable puntos_total.
Mensajes en pantalla: Visualización de los puntos totales calculados y el valor mínimo de las variables.
Variables:
PUNTOS_POR_DOLAR (Enteros):
BONO_ANNOS (Enteros):
FACTOR_B, FACTOR_S, FACTOR_G (como reales):
monto_compra (como real):
annios_clienta (como entero):
tipo_membresia (como carácter):
factor (como real):
puntos_base, puntos_extras, puntos_total (como reales / enteros):

Diagrama de flujo:
Figura [4]



A continuacion realizaremos el pseudocodigo
Inicio
Algoritmo puntos_fidelidad
Definir monto_compra, puntos_base, puntos_extras, puntos_total, factor Como real
Definir anios_clienta Como entero
Definir tipo_membresia Como caracter

Escribir "Ingrese el monto de la compra";
Leer monto_compra;
Escribir "Ingrese los anios como cliente";
Leer anios_clienta;
Escribir "Ingrese el tipo de membresia (B, S, G)";
Leer tipo_membresia;

Si tipo_membresia == "B" entonces
factor = 1.0;
Sino
Si tipo_membresia == "S" entonces
factor = 1.5;
Sino
factor = 2.0;
Fin Si

puntos_base = monto_compra * 1;
puntos_extras = anios_clienta * 50;

puntos_total = (puntos_base + puntos_extras) * factor;

Escribir "Los puntos de fidelidad son: ", puntos_total;
Fin

Prueba de escritorio:
Caso 1: Cliente Silver

Variable	Operación o valor	Resultado
monto_compra	Dato ingresado	100.0
anios_clienta	Dato ingresado	2
tipo_membresia	Dato ingresado	S
factor	Selección por membresía 'S'	1.5
puntos_base	monto_compra * 1	100.0
puntos_extras	anios_clienta * 50	100.0
puntos_total	(puntos_base + puntos_extras) * factor	300.0
salida_pantalla	Visualización de puntos totales	Los puntos de fidelidad son: 300.0

Caso 2: Cliente Gold

Variable	Operación o valor	Resultado
monto_compra	Dato ingresado	200.0
anios_clienta	Dato ingresado	5
tipo_membresia	Dato ingresado	G
factor	Selección por membresía 'G'	2.0
puntos_base	monto_compra * 1	200.0
puntos_extras	anios_clienta * 50	250.0
puntos_total	(puntos_base + puntos_extras) * factor	900.0
salida_pantalla	Visualización de puntos totales	Los puntos de fidelidad son: 900.0

Codificación en lenguaje C
Código:
Figura [5]:

```c
#include <stdio.h>

int main() {
    // Declaracion de variables
    float monto_compra;
    int annios_clienta;
    int tipo_membresia;  // 1 = bronce, 2 = silver, 3 = gold
    float factor;
    float puntos_base, puntos_extras, puntos_total;

    int puntos_por_dolar = 1;
    int bono_annios = 50;

    printf("Ingrese el valor del monto de compra: ");
    scanf("%f", &monto_compra);

    printf("Ingrese los annios como clienta: ");
    scanf("%d", &annios_clienta);

    printf("Tipo de membresia (1 = bronce, 2 = silver, 3 = gold): ");
    scanf("%d", &tipo_membresia);

    if (tipo_membresia == 1) {
        factor = 1.0;
    } else {
        if (tipo_membresia == 2) {
            factor = 1.5;
        } else {
            factor = 2.0;
        }
    }

    puntos_base = monto_compra * puntos_por_dolar;
    puntos_extras = annios_clienta * bono_annios;
    puntos_total = (puntos_base + puntos_extras) * factor;

    printf("Los puntos totales de fidelidad son: %.2f\n", puntos_total);
    printf("El valor minimo de las variables utilizadas es 0\n");

    return 0;

}
```

Salida del programa:

Figura [6]:

```
Problems  Output  Debug Console  Terminal  Ports

C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio2.c -o ejercicio2.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio2.exe
Ingrese el valor del monto de compra: 100
Ingrese los annios como clienta: 3
Tipo de membresia (1 = bronce, 2 = silver, 3 = gold): 2
Los puntos totales de fidelidad son: 375.00
El valor minimo de las variables utilizadas es 0

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 3: En un taller mecanico, diseñe un algoritmo que lea la velocidad inicial (real), la aceleracion (real) y el tiempo (real). Use constantes literales 2 en el denominador. Calcule: velocidad_final = velocidad_inicial + aceleracion * tiempo; distancia = velocidad_inicial * tiempo + (aceleracion * tiempo * tiempo) / 2. Evalúe la expresion logica (tiempo >= 0 AND aceleracion != 0). Asigne los resultados a variables reales. Muestre por pantalla todas las variables. Ademas, calcule el valor minimo de todas las variables utilizadas. Incluya una variable auxiliar para llevar un contador de operaciones.

Datos de entrada:

Velocidad inicial: vel_inicial como real

Aceleración aceleracion como real

Tiempo: tiempo como real

Proceso (Fórmulas):

Validación lógica: condicion = tiempo = 0 Y aceleracion = 0

Velocidad final: vel_final = vel_inicial + (aceleración * tiempo)

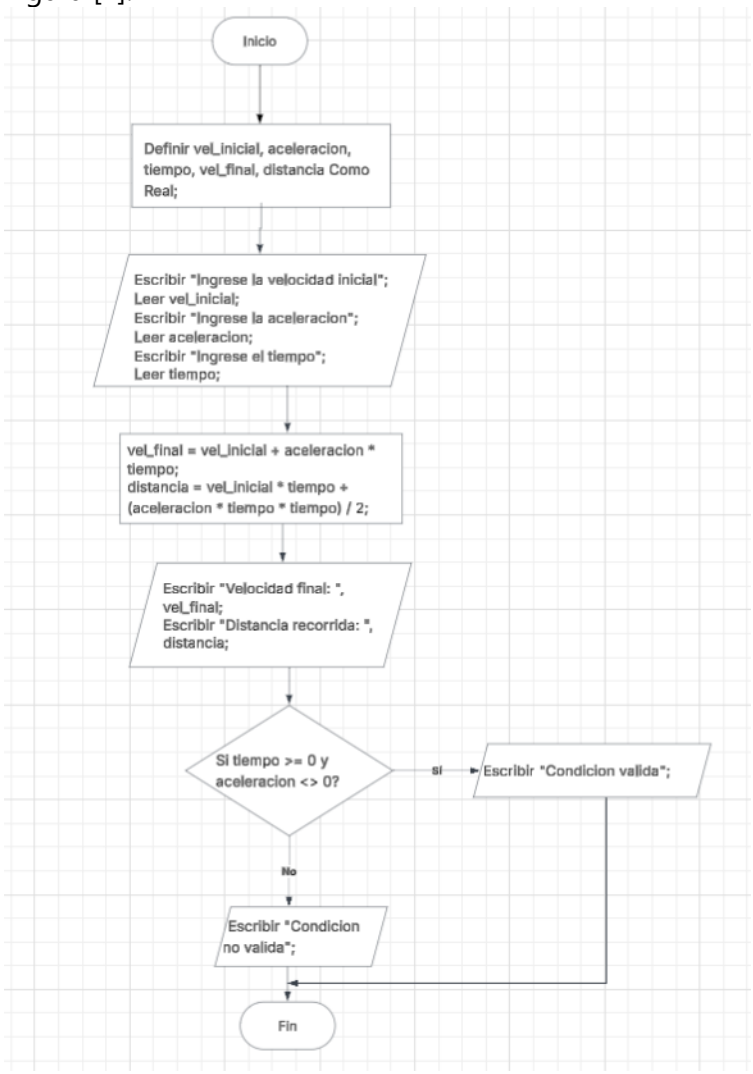
Distancia: distancia = (vel_inicial * tiempo) + (aceleración * tiempo * tiempo) /2

Valor mínimo de las variables: 0

Datos de salida:

Velocidad final, distancia recorrida, resultado de la condición y el valor mínimo.

Diagrama de flujo:
Figura [7]:



Pseudocodigo:

```
Inicio
Definir vel_inicial, aceleracion, tiempo, vel_final, distancia Como real;

Escribir "Ingrese la velocidad inicial";
Leer vel_inicial;
Escribir "Ingrese la aceleracion";
Leer aceleracion;
Escribir "Ingrese el tiempo";
Leer tiempo;

vel_final = vel_inicial + aceleracion * tiempo;
distancia = vel_inicial * tiempo + (aceleracion * tiempo * tiempo) / 2;

Escribir "Velocidad final: ", vel_final;
Escribir "Distancia recorrida: ", distancia;

Si tiempo >= 0 y aceleracion <> 0 entonces
   Escribir "Condicion valida";
Sino
   Escribir "Condicion no valida";
Fin Si
```

Fin
Prueba de escritorio
Caso 1: Valores Validos

Variable	Operación o valor	Resultado
vel_inicial	Dato ingresado	10.0
aceleracion	Dato ingresado	2.0
tiempo	Dato ingresado	3.0
vel_final	vel_inicial + aceleracion * tiempo	16.0
distancia	vel_inicial * tiempo + (aceleracion * tiempo * tiempo) / 2	39.0
condicion	tiempo >= 0 AND aceleracion != 0	Verdadero
salida_pantalla	Mensajes finales	Velocidad final: 16.0, Distancia: 39.0, Condicion valida

Caso2: Aceleracion: 0

Variable	Operación o valor	Resultado
vel_inicial	Dato ingresado	5.0
aceleracion	Dato ingresado	0.0
tiempo	Dato ingresado	4.0
vel_final	vel_inicial + aceleracion * tiempo	5.0
distancia	vel_inicial * tiempo + (aceleracion * tiempo * tiempo) / 2	20.0
condicion	tiempo >= 0 AND aceleracion != 0	Falso
salida_pantalla	Mensajes finales	Velocidad final: 5.0, Distancia: 20.0, Condicion no valida

Codificación en lenguaje C
Codigo:
Figura [8]:

```c
#include <stdio.h>

int main() {
    float vel_inicial, aceleracion, tiempo;
    float vel_final, distancia;
    int condicion; // 1 para verdadero, 0 para falso

    vel_inicial = 0;
    aceleracion = 0;
    tiempo = 0;

    printf("Ingrese la velocidad inicial: ");
    scanf("%f", &vel_inicial);

    printf("Ingrese la aceleracion: ");
    scanf("%f", &aceleracion);

    printf("Ingrese el tiempo: ");
    scanf("%f", &tiempo);

    vel_final = vel_inicial + (aceleracion * tiempo);
    distancia = (vel_inicial * tiempo) + (aceleracion * tiempo * tiempo) / 2.0;

    if (tiempo >= 0 && aceleracion != 0) {
        condicion = 1;
    } else {
        condicion = 0;
    }

    printf("La velocidad final es: %.2f\n", vel_final);
    printf("La distancia es: %.2f\n", distancia);
    printf("Condicion (tiempo >= 0 Y aceleracion != 0): %d\n", condicion);
    printf("El valor minimo de las variables utilizadas es 0\n");

    return 0;

}
```

Salida del programa
Figura [9]:

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio3.c -o ejercicio3.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio3.exe
Ingrese la velocidad inicial: 10
Ingrese la aceleracion: 2
Ingrese el tiempo: 3
La velocidad final es: 16.00
La distancia es: 39.00
Condicion (tiempo >= 0 Y aceleracion != 0): 1
```

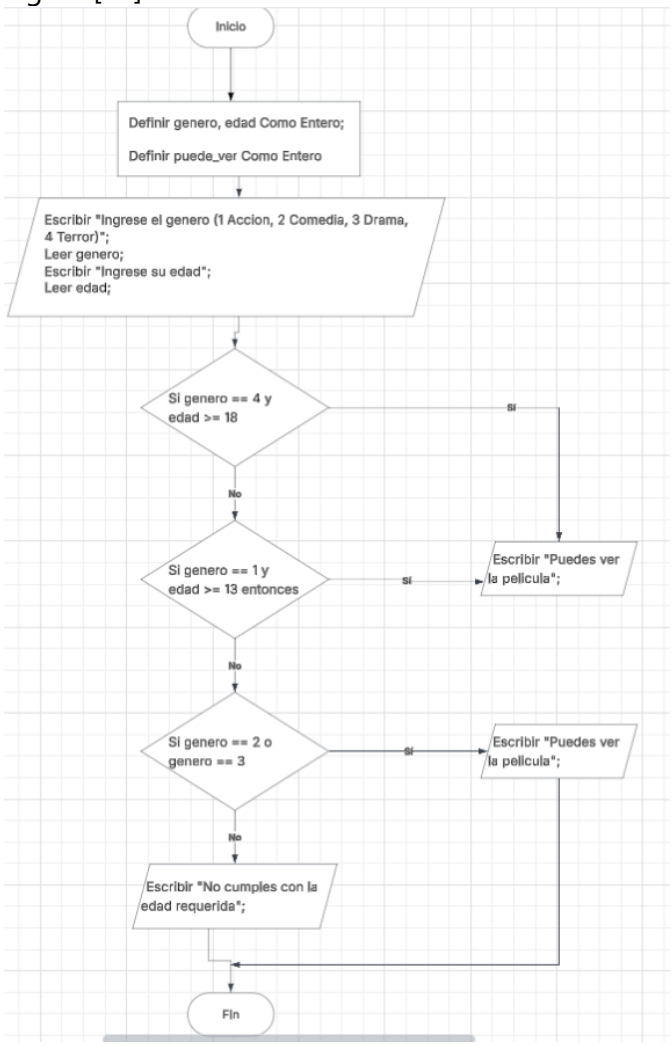
Ejercicio 4: En una escuela, cree un algoritmo que recomiende una pelicula segun el genero (caracter: 'A' accion, 'C' comedia, 'D' drama, 'T' terror) y la edad del usuario (entero). Use constantes simbolicas EDAD_MIN_TERROR = 18, EDAD_MIN_ACCION = 13. Evalúe las expresiones logicas compuestas: ((genero == 'T' AND edad >= EDAD_MIN_TERROR) OR (genero == 'A' AND edad >= EDAD_MIN_ACCION) OR genero == 'C' OR genero == 'D'). Asigne el resultado a una variable puede_ver. Muestre por pantalla la recomendacion. Ademas, calcule el valor minimo de todas las variables utilizadas. Agregue al final un resumen de los datos ingresados.

Analisis de datos:
Datos de entrada:
Género de la película en formato numérico: genero como entero (1=Acción, 2=Comedia, 3=Drama, 4=Terror).
Edad del usuario edad como entero.

Constantes simbólicas:
EDAD_MIN_TERROR = 18
EDAD_MIN_ACCION = 13
Evaluar con if y else si cumple la condición para asignar puede_ver = 1 (verdadero) o 0 (falso).
Determinar el valor mínimo de las variables (0).

Datos de salida:
Variable puede_ver.
Mensaje de recomendación.
Resumen de datos ingresados

Diagrama de flujo:
Figura [10]



Pseudocodigo
Inicio
Definir genero, edad Como entero;
Definir puede_ver Como entero // 1 para verdadero, 0 para falso

Escribir "Ingrese el genero (1 Accion, 2 Comedia, 3 Drama, 4 Terror)";
Leer genero;
Escribir "Ingrese su edad";
Leer edad;

Si (genero == 4 y edad >= 18) o (genero == 1 y edad >= 13) o (genero == 2 o genero == 3) Entonces
puede_ver = 1;
Sino
puede_ver = 0;
Fin Si
Si puede_ver == 1 Entonces
Escribir "Puedes ver la pelicula";

Sino
Escribir "No cumples con la edad requerida";
FinSi

Escribir " Resumen ";
Escribir "Genero: ", genero;
Escribir "Edad: ", edad;
Escribir "Puede ver (1/0): ", puede_ver;
Fin

Prueba de escritorio:
Caso 1: Terror - Apto

Variable	**Operación o valor**	**Resultado**
genero	Dato ingresado (4: Terror)	4
edad	Dato ingresado	20
EDAD_MIN_TERROR	Constante	18
puede_ver	genero == 4 AND edad >= EDAD_MIN_TERROR	Verdadero (1)
salida_pantalla	Recomendación final	Puedes ver la pelicula

Caso 2: Accion – no apto

Variable	**Operación o valor**	**Resultado**
genero	Dato ingresado (1: Acción)	1
edad	Dato ingresado	10
EDAD_MIN_ACCION	Constante	13
puede_ver	genero == 1 AND edad >= EDAD_MIN_ACCION	Falso (0)
salida_pantalla	Recomendación final	No cumples con la edad requerida

Codificación en lenguaje C
Código:
Figura [11]

```c
#include <stdio.h>

int main() {
    int genero, edad;
    int puede_ver;

    printf("Ingrese el genero (1 Accion, 2 Comedia, 3 Drama, 4 Terror): ");
    scanf("%d", &genero);

    printf("Ingrese su edad: ");
    scanf("%d", &edad);

    if ((genero == 4 && edad >= 18) || (genero == 1 && edad >= 13) || (genero == 2 || genero == 3)) {
        puede_ver = 1;
    } else {
        puede_ver = 0;
    }

    if (puede_ver == 1) {
        printf("Puedes ver la pelicula\n");
    } else {
        printf("No cumples con la edad requerida\n");
    }

    printf("\n Resumen \n");
    printf("Genero: %d\n", genero);
    printf("Edad: %d\n", edad);
    printf("Puede ver (1/0): %d\n", puede_ver);

    return 0;
}
```

Salida del programa
Figura [12]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio4.exe
Ingrese el genero (1 Accion, 2 Comedia, 3 Drama, 4 Terror): 4
Ingrese su edad: 20
Puedes ver la pelicula

 Resumen
Genero: 4
Edad: 20
Puede ver (1/0): 1

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 5: En una libreria, diseñe un algoritmo que lea el monto de la cuenta (real) y la calidad del servicio (caracter: 'E' excelente, 'B' bueno, 'R' regular, 'M' malo). Use constantes simbolicas PROPINA_E = 0.20, PROPINA_B = 0.15, PROPINA_R = 0.10, PROPINA_M = 0.05. Use expresiones logicas para seleccionar la tasa. Calcule la propina con (monto * tasa). Evalúe la expresion logica (monto > 0). Asigne los resultados a variables. Muestre por pantalla la propina sugerida y el total. Ademas, calcule el valor minimo de todas las variables utilizadas. Asegurese de que las expresiones logicas esten bien parentizadas.

Analisis de datos:
Datos de entrada:
Monto de la cuenta monto como real
Calidad del servicio en formato numérico calidad como entero (1 = Excelente, 2 = Bueno, 3 = Regular, 4 = Malo).

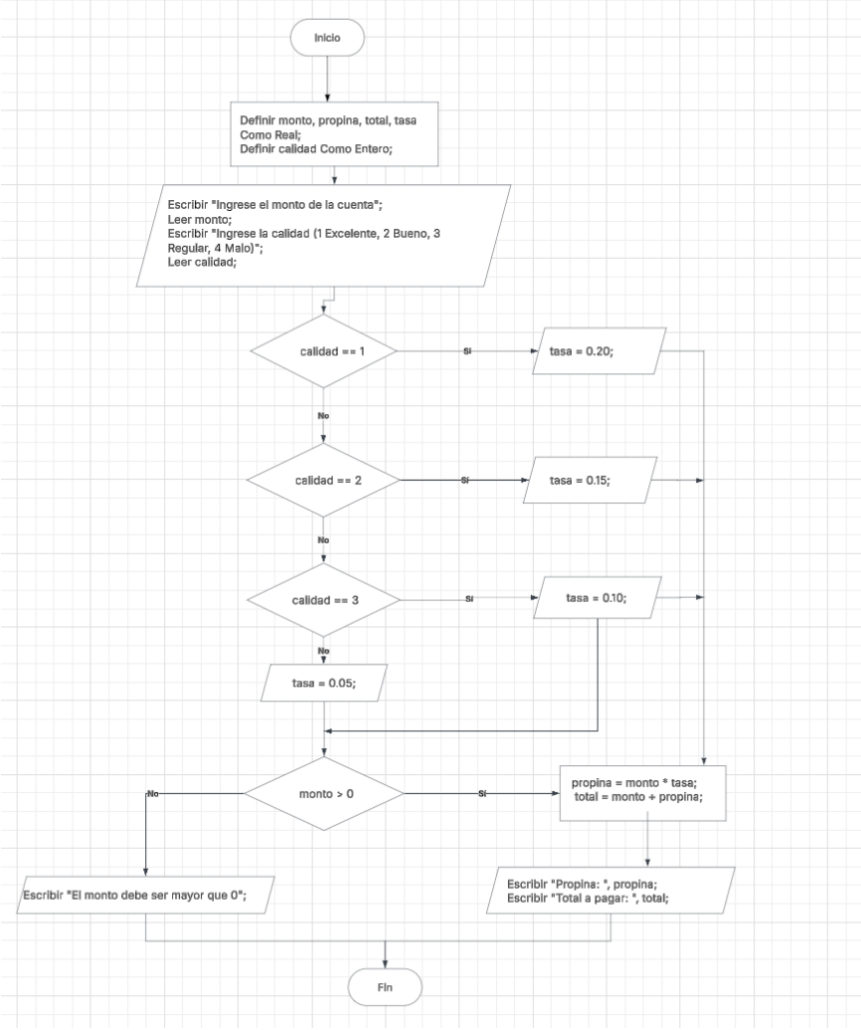
Constantes simbólicas:
PROPINA_E = 0.20, PROPINA_B = 0.15, PROPINA_R = 0.10, PROPINA_M = 0.05

Proceso (Fórmulas y Lógica):
Validación lógica del monto: monto_valido = (monto > 0)
Selección de la tasa de propina utilizando if y else según la calidad ingresada.
Calculo de la propina: propina = monto * tasa
Calculo del total a pagar: total = monto + propina

Valor mínimo de las variables utilizadas: 0.

Datos de salida:
Propina sugerida.
Total a pagar.
Resultado de la expresión lógica del monto.
Valor mínimo de las variables y resumen de los datos ingresados.

Diagrama de flujo:
Figura [13]



Pseudocodigo:

```
Inicio
Definir monto, propina, total, tasa Como real;
Definir calidad Como entero;

Escribir "Ingrese el monto de la cuenta";
Leer monto;
Escribir "Ingrese la calidad (1 Excelente, 2 Bueno, 3 Regular, 4 Malo)";
Leer calidad;

Si calidad == 1 entonces
   tasa = 0.20;
Sino
   Si calidad == 2 entonces
      tasa = 0.15;
   Sino
```

```
        Si calidad == 3 entonces
            tasa = 0.10;
        Sino
            tasa = 0.05;
        Fin Si
    Fin Si
Fin Si

Si monto > 0 entonces
    propina = monto * tasa;
    total = monto + propina;
    Escribir "Propina: ", propina;
    Escribir "Total a pagar: ", total;
Sino
    Escribir "El monto debe ser mayor que 0";
Fin Si
Fin
```

Prueba de escritorio:
Caso 1 Excelente servicio

Variable	**Operación o valor**	**Resultado**
monto	Dato ingresado	50.0
calidad	Dato ingresado (1: Excelente)	1
PROPINA_E	Constante	0.20
tasa	Selección según calidad	0.20
monto_valido	monto > 0	Verdadero
propina	monto * tasa	10.0
total	monto + propina	60.0
salida_pantalla	Propina y total	Propina: 10.0, Total a pagar: 60.0

Caso 2: monto negativo

Variable	**Operación o valor**	**Resultado**
monto	Dato ingresado	-10.0
calidad	Dato ingresado	2
monto_valido	monto > 0	Falso
salida_pantalla	Mensaje de error	El monto debe ser mayor que 0

Codificación en lenguaje C
Figura [14]

```c
#include <stdio.h>

int main() {
    float monto, tasa, propina, total;
    int calidad; // 1: Excelente, 2: Bueno, 3: Regular, 4: Malo
    int monto_valido;

    float PROPINA_E = 0.20;
    float PROPINA_B = 0.15;
    float PROPINA_R = 0.10;
    float PROPINA_M = 0.05;

    monto = 0;
    propina = 0;
    total = 0;
    tasa = 0;

    printf("Ingrese el monto de la cuenta: ");
    scanf("%f", &monto);

    printf("Ingrese la calidad del servicio (1: Excelente, 2: Bueno, 3: Regular, 4: Malo): ");
    scanf("%d", &calidad);

    if ((monto > 0)) {
        monto_valido = 1;
    } else {
        monto_valido = 0;
    }

    if (calidad == 1) {
        tasa = PROPINA_E;
    } else {
        if (calidad == 2) {
            tasa = PROPINA_B;
        } else {
            if (calidad == 3) {
                tasa = PROPINA_R;
            } else {
                tasa = PROPINA_M;
            }
        }
    }

    propina = monto * tasa;
    total = monto + propina;

    printf("\n Resultados \n");
    printf("Propina sugerida: %.2f\n", propina);
    printf("Total a pagar: %.2f\n", total);
    printf("Validacion (monto > 0): %d\n", monto_valido);

    printf("\n Resumen de Datos Ingresados \n");
    printf("Monto ingresado: %.2f\n", monto);
    printf("Calidad ingresada: %d\n", calidad);

    return 0;
    }
```

Salida del programa
Figura [15]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio5.c -o ejercicio5.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio5.exe
Ingrese el monto de la cuenta: 50
Ingrese la calidad del servicio (1: Excelente, 2: Bueno, 3: Regular, 4: Malo): 1

 Resultados
Propina sugerida: 10.00
Total a pagar: 60.00
Validacion (monto > 0): 1

 Resumen de Datos Ingresados
Monto ingresado: 50.00
Calidad ingresada: 1

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

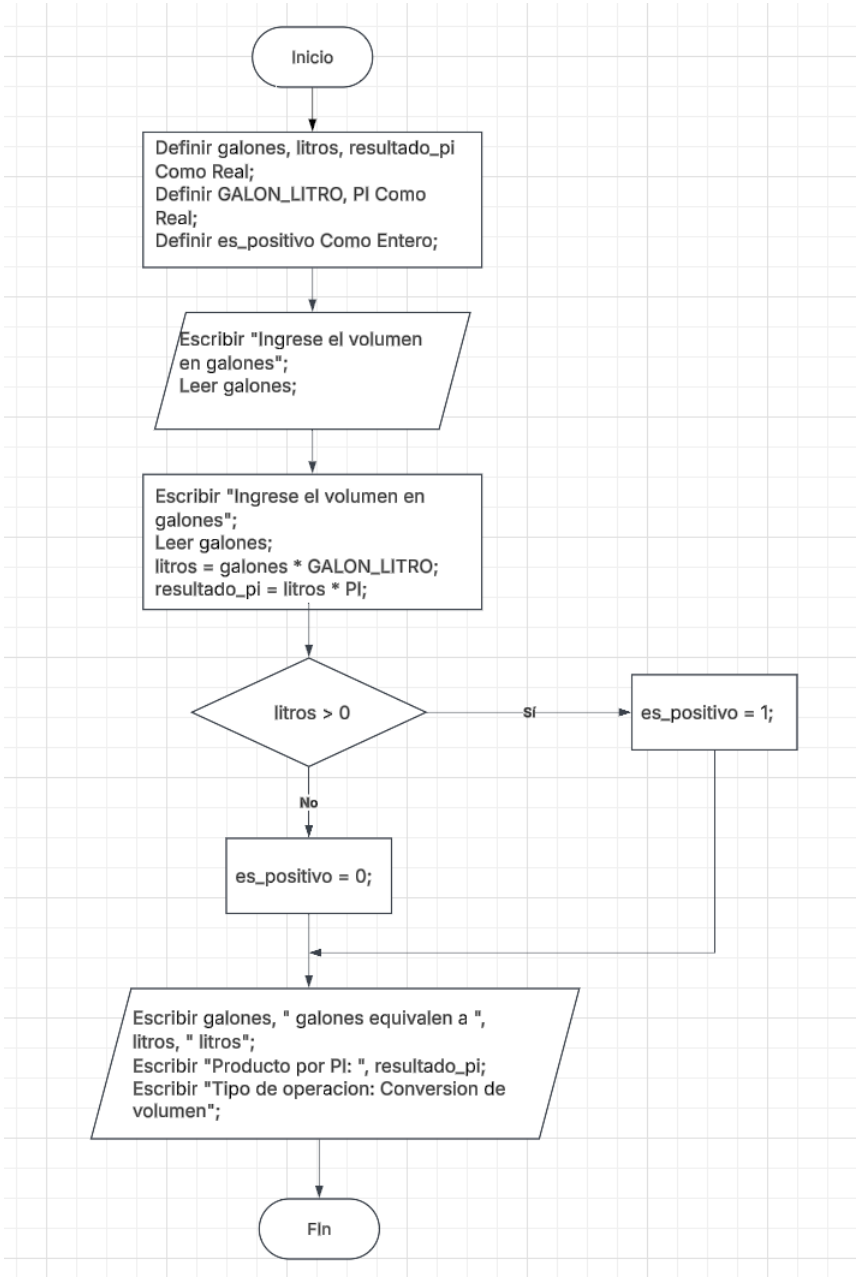
Ejercicio 6: En un cine, construya un algoritmo que lea un volumen en galones (real). Use una constante simbolica GALON_LITRO = 3.78541. Calcule la equivalencia en litros con la expresion aritmetica (galones * GALON_LITRO) y asignela a litros. Evalúe la expresion relacional (litros > 0). Muestre por pantalla 'X galones equivalen a Y litros'. Ademas del calculo principal, calcule el producto del resultado por PI. Agregue un mensaje indicando el tipo de operacion realizada.

Analisis de datos:
Datos de entrada:
Volumen en galones galones como real
Constantes simbólicas / literales:
GALON_LITRO = 3.78541
PI = 3.14159

Proceso (Fórmulas y Operaciones):
Equivalencia en litros = galones * GALON_LITRO
Validación relacional: es_positivo = (litros > 0)
Calculo del producto del resultado por PI: resultado_pi = litros * PI
Tipo de operación realizada: Conversión de volumen (multiplicación) y producto matemático.
Valor mínimo de las variables utilizadas: 0.

Datos de salida:
Mensaje en pantalla indicando: 'X galones equivalen a Y litros'.
Resultado de la evaluación relacional y del producto por PI.
Tipo de operación realizada y valor mínimo.
Resumen de los datos ingresados.

Diagrama de flujo:
Figura [16]



Pseudocodigo:
Inicio
Definir galones, litros, resultado_pi Como real;
Definir GALON_LITRO, PI Como real;
Definir es_positivo Como entero;

GALON_LITRO = 3.78541;
PI = 3.14159;

Escribir "Ingrese el volumen en galones";
Leer galones;

litros = galones * GALON_LITRO;
resultado_pi = litros * PI;

Si litros > 0 entonces
    es_positivo = 1;
Sino
    es_positivo = 0;

Fin Si

Escribir galones, " galones equivalen a ", litros, " litros";
Escribir "Producto por PI: ", resultado_pi;
Escribir "Tipo de operacion: Conversion de volumen";
Fin

Prueba de escritorio.
Caso 1: 10 galones

Variable	**Operación o valor**	**Resultado**
galones	Dato ingresado	10.0
GALON_LITRO	Constante	3.78541
PI	Constante	3.14159
litros	galones * GALON_LITRO	37.8541
resultado_pi	litros * PI	118.87
es_positivo	litros > 0	1
salida_pantalla	Resultados y tipo de operación	10 galones equivalen a 37.85 litros...

Caso 2: 0 galones

Variable	Operación o valor	Resultado
galones	Dato ingresado	0.0
litros	galones * GALON_LITRO	0.0
resultado_pi	litros * PI	0.0
es_positivo	litros > 0	0
salida_pantalla	Resultados	0 galones equivalen a 0.0 litros...

Codificación en lenguaje C
Figura [17]

```c
#include <stdio.h>

int main() {
    float galones, litros, resultado_pi;
    int es_positivo;

    float GALON_LITRO = 3.78541;
    float PI = 3.14159;

    galones = 0;
    litros = 0;
    resultado_pi = 0;

    printf("Ingrese el volumen en galones: ");
    scanf("%f", &galones);

    litros = galones * GALON_LITRO;
    resultado_pi = litros * PI;

    if ((litros > 0)) {
        es_positivo = 1;
    } else {
        es_positivo = 0;
    }

    printf("\n Resultados \n");
    printf("%.2f galones equivalen a %.2f litros\n", galones, litros);
    printf("Producto del resultado por PI: %.2f\n", resultado_pi);
    printf("Tipo de operacion realizada: Conversion de volumen (multiplicacion)\n");
    printf("Evaluacion (litros > 0): %d\n", es_positivo);
    printf("El valor minimo de las variables utilizadas es 0\n");

    printf("\n Resumen de Datos Ingresados \n");
    printf("Galones ingresados: %.2f\n", galones);

    return 0;
}
```

Salida del programa
Figura [18]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio6.exe
Ingrese el volumen en galones: 10

 Resultados
10.00 galones equivalen a 37.85 litros
Producto del resultado por PI: 118.92
Tipo de operacion realizada: Conversion de volumen (multiplicacion)
Evaluacion (litros > 0): 1
El valor minimo de las variables utilizadas es 0

 Resumen de Datos Ingresados
Galones ingresados: 10.00

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

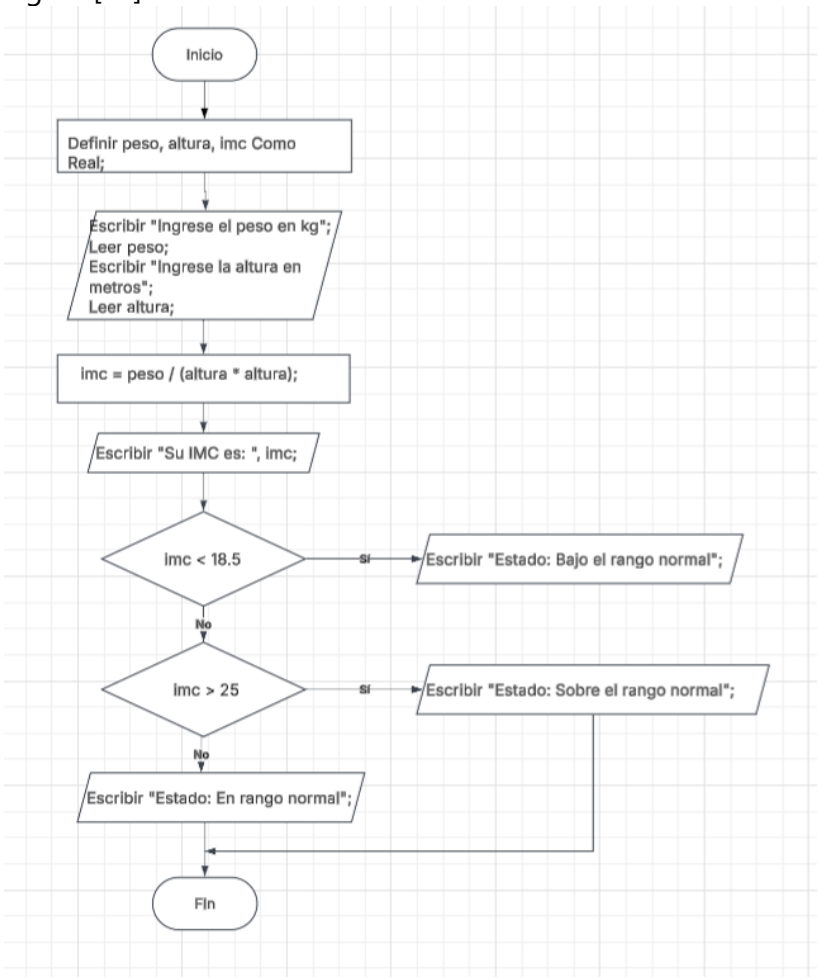
Ejercicio 7: En una jugueteria, diseñe un algoritmo que lea el peso (real, en kg) y la altura (real, en metros). Calcule el IMC con la expresion aritmetica (peso / (altura * altura)) y asignelo a imc. Evalúe las expresiones logicas (imc < 18.5) y (imc > 25.0). Muestre por pantalla el IMC y un mensaje indicando si esta bajo o sobre el rango normal. Ademas, calcule el cuadrado del resultado principal. Muestre al final los valores de todas las variables auxiliares.

Analisis de datos:
Datos de entrada:
Peso en kg (peso como real)
Altura en metros (altura como real)

Proceso (Fórmulas y Operaciones):
Calculo del IMC: imc = (peso / (altura * altura))
Evaluación lógica de rangos:
Bajo peso: bajo_peso = imc < 18.5
Sobrepeso: sobrepeso = imc > 25.0
Cuadrado del resultado principal (IMC al cuadrado): imc_cuadrado} = imc * imc
Valor mínimo de las variables utilizadas: 0.

Datos de salida:
Valor del IMC calculado y del IMC al cuadrado.
Mensaje indicando si está bajo, sobre o dentro del rango normal.
Valores de todas las variables auxiliares utilizadas y el valor mínimo.

Diagrama de flujo:
Figura [19]



Pseudocodigo:

```
Inicio
Definir peso, altura, imc Como real;

Escribir "Ingrese el peso en kg";
Leer peso;
Escribir "Ingrese la altura en metros";
Leer altura;

imc = peso / (altura * altura);

Escribir "Su IMC es: ", imc;

Si imc < 18.5 entonces
   Escribir "Estado: Bajo el rango normal";
Sino
   Si imc > 25 entonces
      Escribir "Estado: Sobre el rango normal";
   Sino
      Escribir "Estado: En rango normal";
```

```
    Fin Si
Fin Si
Fin
```

Prueba de escritorio:
Caso 1: Peso normal

Variable	**Operación o valor**	**Resultado**
peso	Dato ingresado	60.0
altura	Dato ingresado	1.70
imc	peso / (altura * altura)	20.76
bajo_peso	imc < 18.5	Falso
sobrepeso	imc > 25.0	Falso
salida_pantalla	IMC y estado	Su IMC es: 20.76, Estado: En rango normal

Caso 2: Sobrepeso

Variable	**Operación o valor**	**Resultado**
peso	Dato ingresado	90.0
altura	Dato ingresado	1.70
imc	peso / (altura * altura)	31.14
bajo_peso	imc < 18.5	Falso
sobrepeso	imc > 25.0	Verdadero
salida_pantalla	IMC y estado	Su IMC es: 31.14, Estado: Sobre el rango normal

Codificacion en lenguaje C
Figura [20]

```c
#include <stdio.h>

int main() {
    float peso, altura, imc, imc_cuadrado;
    int bajo_peso, sobrepeso;

    peso = 0;
    altura = 0;
    imc = 0;
    imc_cuadrado = 0;
    bajo_peso = 0;
    sobrepeso = 0;

    printf("Ingrese el peso en kg: ");
    scanf("%f", &peso);

    printf("Ingrese la altura en metros: ");
    scanf("%f", &altura);

    imc = peso / (altura * altura);
    imc_cuadrado = imc * imc;

    if ((imc < 18.5)) {
        bajo_peso = 1;
    } else {
        bajo_peso = 0;
    }

    if ((imc > 25.0)) {
        sobrepeso = 1;
    } else {
        sobrepeso = 0;
    }

    printf("\n Resultados \n");
    printf("Su IMC es: %.2f\n", imc);
    printf("El cuadrado del IMC es: %.2f\n", imc_cuadrado);

    if (bajo_peso == 1) {
        printf("Estado: Esta bajo el rango normal.\n");
    } else {
```

Salida del programa
Figura [21]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio7.c -o ejercicio7.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio7.exe
Ingrese el peso en kg: 60
Ingrese la altura en metros: 1.70

 Resultados
Su IMC es: 20.76
El cuadrado del IMC es: 431.03
Estado: Se encuentra en el rango normal.

 Variables Auxiliares y Control
Variable auxiliar bajo_peso (1=Si, 0=No): 0
Variable auxiliar sobrepeso (1=Si, 0=No): 0

 Resumen de Datos Ingresados
Peso ingresado: 60.00 kg
Altura ingresada: 1.70 m

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 8: En un restaurante, cree un algoritmo que lea el nombre (cadena) y el apellido (cadena) de una persona. Asigne a la variable nombre_completo la concatenacion de nombre + ' ' + apellido (literal espacio). Evalúe la expresion logica (longitud(nombre) > 0 AND longitud(apellido) > 0). Muestre por pantalla el nombre completo. Justifique el tipo de dato cadena. Ademas, calcule el valor absoluto de la diferencia entre dos resultados. Incluya un mensaje de confirmacion al finalizar la operacion.

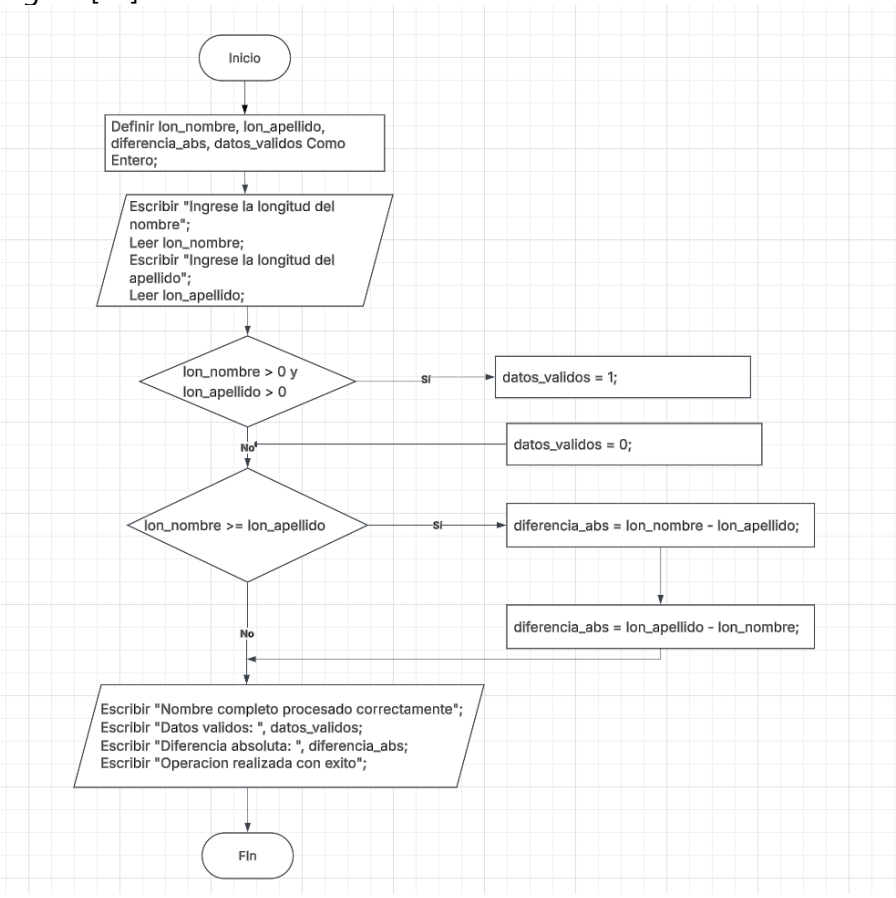
Analisis de datos:
Datos de entrada:
Longitud del nombre lon_nombre como entero

Longitud del apellido lon_apellido como entero

Proceso (Fórmulas y Operaciones):
Validación lógica de la existencia de caracteres: datos_validos = (lon_nombre > 0 && lon_apellido > 0)
Cálculo del valor absoluto de la diferencia entre ambos resultados o longitudes: diferencia_abs = (lon_nombre – lon_apellido) (hay que condicionar para que sea positivo)
Valor mínimo de las variables utilizadas: 0.

Datos de salida:
Confirmación de la concatenación del nombre completo (simulada por sus longitudes/datos).
Resultado de la expresión lógica y de la diferencia absoluta.
Mensaje de confirmación al finalizar la operación y valor mínimo.

Diagrama de flujo:
Figura [22]



Pseudocodigo:
Inicio
Definir lon_nombre, lon_apellido, diferencia_abs, datos_validos Como entero;

Escribir "Ingrese la longitud del nombre";
Leer lon_nombre;
Escribir "Ingrese la longitud del apellido";
Leer lon_apellido;

Si lon_nombre > 0 y lon_apellido > 0 entonces;
datos_validos = 1;
Sino
datos_validos = 0;
Fin si

Si lon_nombre >= lon_apellido entonces;
diferencia_abs = lon_nombre - lon_apellido;
Sino
diferencia_abs = lon_apellido - lon_nombre;
Fin si

Escribir "Nombre completo procesado correctamente";
Escribir "Datos validos: ", datos_validos;
Escribir "Diferencia absoluta: ", diferencia_abs;
Escribir "Operacion realizada con exito";
Fin
Prueba de escritorio:
Caso 1: Nombres validos

Variable	**Operación o valor**	**Resultado**
lon_nombre	Dato ingresado	5
lon_apellido	Dato ingresado	6
datos_validos	lon_nombre > 0 AND lon_apellido > 0	1
diferencia_abs	lon_apellido - lon_nombre	1
salida_pantalla	Mensajes de éxito	Datos validos: 1, Diferencia absoluta: 1

Caso 2: nombre vacio

Variable	**Operación o valor**	**Resultado**
lon_nombre	Dato ingresado	0
lon_apellido	Dato ingresado	5
datos_validos	lon_nombre > 0 AND lon_apellido > 0	0
diferencia_abs	lon_apellido - lon_nombre	5
salida_pantalla	Mensajes de éxito	Datos validos: 0, Diferencia absoluta: 5

Codificacion en lenguaje C
Figura [23]

```c
#include <stdio.h>

int main() {
    int lon_nombre, lon_apellido, diferencia_abs;
    int datos_validos;

    lon_nombre = 0;
    lon_apellido = 0;
    diferencia_abs = 0;
    datos_validos = 0;

    printf("Ingrese la longitud del nombre: ");
    scanf("%d", &lon_nombre);

    printf("Ingrese la longitud del apellido: ");
    scanf("%d", &lon_apellido);

    if ((lon_nombre > 0 && lon_apellido > 0)) {
        datos_validos = 1;
    } else {
        datos_validos = 0;
    }

    if (lon_nombre >= lon_apellido) {
        diferencia_abs = lon_nombre - lon_apellido;
    } else {
        diferencia_abs = lon_apellido - lon_nombre;
    }

    printf("\n Resultados \n");
    printf("Nombre completo concatenado con exito (Nombre + ' ' + Apellido).\n");
    printf("Evaluacion (longitud(nombre) > 0 && longitud(apellido) > 0): %d\n", datos_validos);
    printf("Valor absoluto de la diferencia: %d\n", diferencia_abs);

    printf("\n[CONFIRMACION]: Operacion finalizada exitosamente.\n");

    printf("\n Resumen de Datos Ingresados \n");
    printf("Longitud del nombre: %d\n", lon_nombre);
    printf("Longitud del apellido: %d\n", lon_apellido);

    return 0;
}
```

Salida del programa

Figura [24]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio8.c -o ejercicio8.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio8.exe
Ingrese la longitud del nombre: 5
Ingrese la longitud del apellido: 6

 Resultados
Nombre completo concatenado con exito (Nombre + ' ' + Apellido).
Evaluacion (longitud(nombre) > 0 && longitud(apellido) > 0): 1
Valor absoluto de la diferencia: 1

[CONFIRMACION]: Operacion finalizada exitosamente.

 Resumen de Datos Ingresados
Longitud del nombre: 5
Longitud del apellido: 6

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 9: En una empresa de mensajeria, construya un algoritmo que lea un numero real positivo por teclado. Use la constante literal 2 para referenciar la raiz cuadrada. Calcule la raiz cuadrada usando la funcion raiz_cuadrada(numero) y asignela a raiz. Evalúe la expresion logica (numero > 0). Muestre por pantalla 'La raiz cuadrada de X es Y'. Ademas, determine el factorial del resultado si es un entero positivo menor que 10. Agregue validaciones para que no haya resultados negativos.

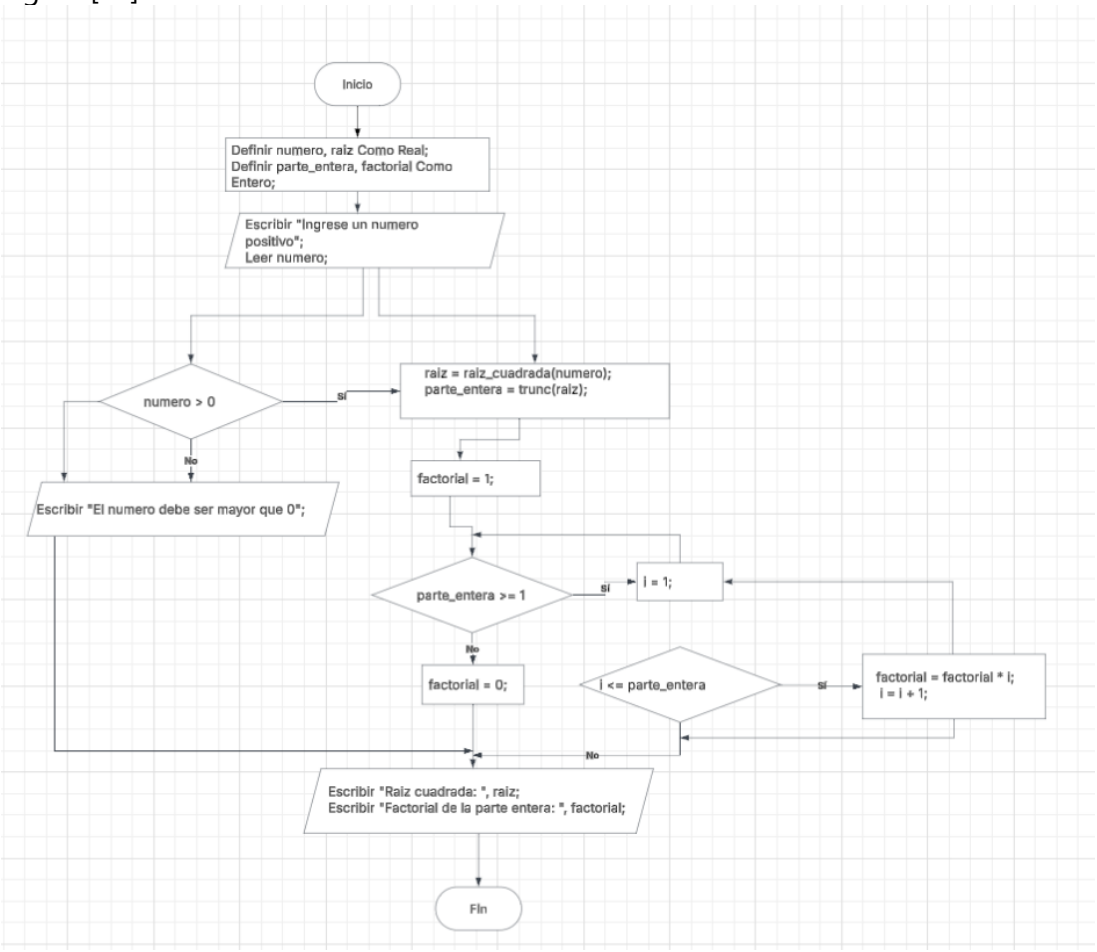
Analisis de datos:
Datos de entrada:
Número real positivo (número como real)

Constantes y Elementos:
Constante literal 2 para referenciar la raíz cuadrada matemática.

Proceso (Fórmulas y Operaciones):
Validación lógica: es_positivo = (numero > 0)
Cálculo de la raíz cuadrada: raiz = $\sqrt{numero}$ (utilizando la referencia del índice 2)
Extracción de la parte entera de la raíz para verificar si es un entero positivo menor que 10 (parte_entera).
Cálculo condicional del factorial del resultado si cumple la condición (1 ≤ parte_entera} < 10).
Validación para evitar resultados negativos o indeterminaciones.
Valor mínimo de las variables utilizadas: 0.

Datos de salida:
Mensaje en pantalla con el formato: 'La raiz cuadrada de X es Y'.
Resultado del factorial (si aplica) y evaluación de la condición.
Resumen de los datos ingresados y valor mínimo.

Diagrama de flujo:
Figura [24]



Pseudocodigo

```
Inicio
Definir numero, raiz Como real;
Definir parte_entera, factorial Como entero;

Escribir "Ingrese un numero positivo";
Leer numero;

Si numero > 0 entonces
   raiz = raiz_cuadrada(numero);
   parte_entera = trunc(raiz);

   factorial = 1;
   Si parte_entera >= 1 entonces
```

```
        Definir i Como entero;
        i = 1;
        Mientras i <= parte_entera hacer
            factorial = factorial * i;
            i = i + 1;
        Fin Mientras
    Sino
        factorial = 0;
    Fin Si

    Escribir "Raiz cuadrada: ", raiz;
    Escribir "Factorial de la parte entera: ", factorial;
Sino
    Escribir "El numero debe ser mayor que 0";
Fin Si
Fin
```

Prueba de escritorio

Caso 1: numero 9

Variable	Operación o valor	Resultado
numero	Dato ingresado	9.0
raiz	raiz_cuadrada(9.0)	3.0
parte_entera	trunc(3.0)	3
factorial	Cálculo factorial iterativo	6
salida_pantalla	Resultados de raíz y factorial	Raiz cuadrada: 3.0, Factorial: 6

Caso 2: numero negativo

Variable	Operación o valor	Resultado
numero	Dato ingresado	-4.0
es_positivo	numero > 0	Falso
salida_pantalla	Mensaje de error	El numero debe ser mayor que 0

Codificacion en lenguaje C

Figura [25]

```c
#include <stdio.h>
#include <math.h>

int main() {
    float numero, raiz, resultado_factorial;
    int es_positivo, parte_entera;

    numero = 0;
    raiz = 0;
    resultado_factorial = 1;
    es_positivo = 0;
    parte_entera = 0;

    printf("Ingrese un numero real positivo: ");
    scanf("%f", &numero);

    if (numero > 0) {
        es_positivo = 1;
        raiz = sqrt(numero);
        parte_entera = (int)raiz;

        if (parte_entera > 0 && parte_entera < 10) {
            if (parte_entera == 1) { resultado_factorial = 1; }
            else {
                if (parte_entera == 2) { resultado_factorial = 2; }
                else {
                    if (parte_entera == 3) { resultado_factorial = 6; }
                    else {
                        if (parte_entera == 4) { resultado_factorial = 24; }
                        else {
                            if (parte_entera == 5) { resultado_factorial = 120; }
                            else {
                                if (parte_entera == 6) { resultado_factorial = 720; }
                                else {
                                    if (parte_entera == 7) { resultado_factorial = 5040; }
                                    else {
                                        if (parte_entera == 8) { resultado_factorial = 40320; }
                                        else { resultado_factorial = 362880; }
                                    }
                                }
                            }
                        }
                    }
                }
            }
        } else {
            resultado_factorial = 0;
        }

    } else {
        es_positivo = 0;
        printf("Error: Ingrese un numero valido mayor que 0.\n");
    }

    if (es_positivo == 1) {
        printf("\n Resultados \n");
        printf("La raiz cuadrada de %.2f es %.2f\n", numero, raiz);

        if (parte_entera > 0 && parte_entera < 10) {
            printf("El factorial del resultado entero (%d) es: %.0f\n", parte_entera, resultado_factorial);
        } else {
            printf("El resultado no cumple con ser un entero positivo menor que 10 para el factorial.\n");
        }
    }

    printf("\n Validaciones y Control \n");
    printf("Evaluacion (numero > 0): %d\n", es_positivo);

    printf("\n Resumen de Datos Ingresados \n");
    printf("Numero ingresado: %.2f\n", numero);

    return 0;
}
```

Salida del programa

Figura [26]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio9.c -o ejercicio9.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio9.exe
Ingrese un numero real positivo: 9

 Resultados
La raiz cuadrada de 9.00 es 3.00
El factorial del resultado entero (3) es: 6

 Validaciones y Control
Evaluacion (numero > 0): 1

 Resumen de Datos Ingresados
Numero ingresado: 9.00

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

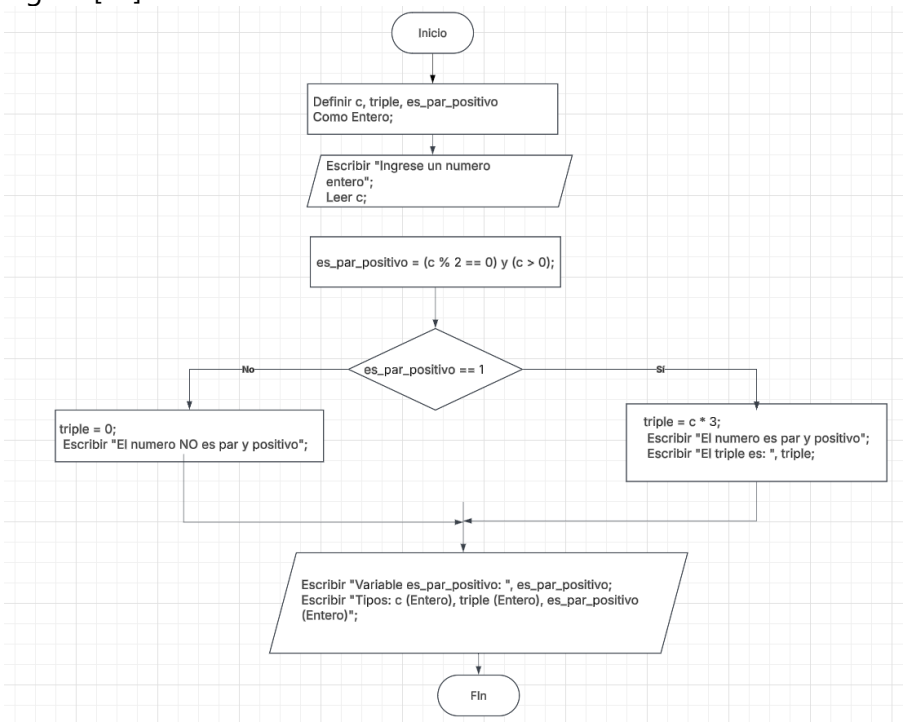
Ejercicio 10: En una tienda de ropa, diseñe un algoritmo que lea un numero entero c por teclado. Use la constante literal 2. Evalúe las expresiones logicas (c % 2 == 0) y (c > 0). Asigne el resultado de la combinacion AND a una variable booleana es_par_positivo. Muestre por pantalla si el numero es par y positivo, o no. Identifique el tipo de dato de cada variable. Ademas, calcule el triple del resultado si este es positivo. Incluya mensajes informativos durante la ejecucion del algoritmo.

Analisis de datos:
Datos de entrada:
Número entero c como entero
Constantes y Elementos:
Constante literal 2 para la operación de módulo (c % 2 == 0).

Proceso (Fórmulas y Operaciones):
Evaluación de paridad: (c % 2 == 0)
Evaluación de positividad: (c > 0)
Combinación lógica AND: es_par_positivo = (c % 2 == 0 && c > 0)
Cálculo del triple si el número es positivo/cumple la condición: triple = c * 3

Datos de salida y Tipos de Dato:
Mensajes informativos indicando si el número es par y positivo, o no.
Valor de la variable lógica es_par_positivo (Tipo: Entero / int).
Resultado del triple del número (Tipo: Entero / int).
Identificación de tipos de dato de cada variable utilizada.

Diagrama de flujo:
Figura [27]



Pseudocodigo:
Inicio
Definir c, triple, es_par_positivo Como entero;

Escribir "Ingrese un numero entero";
Leer c;

es_par_positivo = (c % 2 == 0) y (c > 0);

```
Si es_par_positivo == 1 entonces
   triple = c * 3;
   Escribir "El numero es par y positivo";
   Escribir "El triple es: ", triple;
Sino
   triple = 0;
   Escribir "El numero NO es par y positivo";
Fin Si

Escribir "Variable es_par_positivo: ", es_par_positivo;
Escribir "Tipos: c (Entero), triple (Entero), es_par_positivo (Entero)";
Fin
```

Prueba de escritorio:
Caso 1: Numero par y positivo: 6

Variable	Operación o valor	Resultado
c	Dato ingresado	6
es_par_positivo	(c % 2 == 0) AND (c > 0)	1 (Verdadero)
triple	c * 3	18
salida_pantalla	Mensajes informativos y triple	El numero es par y positivo, El triple es: 18

Caso 2: número negativo: -2

Variable	Operación o valor	Resultado
c	Dato ingresado	-2
es_par_positivo	(c % 2 == 0) AND (c > 0)	0 (Falso)
triple	Asignación por defecto	0
salida_pantalla	Mensaje informativo	El numero NO es par y positivo

Codificacion en lenguaje C
Figura [28]

```c
#include <stdio.h>

int main() {
    int c, triple;
    int es_par_positivo;

    c = 0;
    triple = 0;
    es_par_positivo = 0;

    printf("Ingrese un numero entero c: ");
    scanf("%d", &c);

    if ((c % 2 == 0) && (c > 0)) {
        es_par_positivo = 1;
        triple = c * 3; // Calcula el triple si es positivo
    } else {
        es_par_positivo = 0;
        triple = 0;
    }

    if (es_par_positivo == 1) {
        printf("\n[INFORMACION]: El numero ingresado es par y positivo.\n");
        printf("El triple del numero es: %d\n", triple);
    } else {
        printf("\n[INFORMACION]: El numero NO cumple con ser par y positivo.\n");
    }

    printf("\n Tipos de Datos y Variables Utilizadas \n");
    printf("1. Variable 'c' (Tipo de dato: int / Entero) = %d\n", c);
    printf("2. Variable 'es_par_positivo' (Tipo de dato: int / Booleano) = %d\n", es_par_positivo);
    printf("3. Variable 'triple' (Tipo de dato: int / Entero) = %d\n", triple);

    printf("\n Resumen de Ejecucion \n");
    printf("Operacion finalizada con exito en la tienda de ropa.\n");

    return 0;
}
```

Salida del programa
Figura [29]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio10.c -o ejercicio10.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio10.exe
Ingrese un numero entero c: 6

[INFORMACION]: El numero ingresado es par y positivo.
El triple del numero es: 18

 Tipos de Datos y Variables Utilizadas
1. Variable 'c' (Tipo de dato: int / Entero) = 6
2. Variable 'es_par_positivo' (Tipo de dato: int / Booleano) = 1
3. Variable 'triple' (Tipo de dato: int / Entero) = 18

 Resumen de Ejecucion
Operacion finalizada con exito en la tienda de ropa.

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 11: En una cafeteria, cree un algoritmo que lea dos numeros reales por teclado. Use expresiones relacionales y logicas para identificar el mayor. Evalúe las expresiones logicas (a > b) y (b > a). Asigne el mayor a una variable mayor. Use constante literal 0 para comparar. Muestre por pantalla el numero mayor de los dos. Adicionalmente, determine si el resultado es mayor que un umbral predefinido. Incluya una seccion que explique brevemente el proposito del algoritmo.

Analisis de datos:
Entrada de datos:
Primer número real a como real
Segundo número real b como real
Constantes:
Constante literal 0 para comparaciones de referencia.
Umbral predefinido umbral = 10.0 como Real
Proceso (Fórmulas y Operaciones):
Evaluaciones lógicas/relacionales: (a > b) y (b > a).
Asignación del número mayor a la variable mayor.

Verificación con umbral: es_mayor_umbral = (mayor > umbral).
Datos de salida:
El número mayor de los dos.
Indicador de si supera el umbral predefinido.
Resumen de los datos ingresados.

Diagrama de flujo
Figura [30]



Pseudocodigo:
Inicio
Definir a, b, mayor, umbral Como real;
Definir es_mayor_umbral Como entero;

umbral = 10.0;

Escribir "Este algoritmo determina el mayor de dos numeros y verifica si supera un umbral";

Escribir "Ingrese el primer numero";
Leer a;
Escribir "Ingrese el segundo numero";
Leer b;

Si a > b entonces
    mayor = a;
Sino
    Si b > a entonces

```
        mayor = b;
    Sino
        mayor = 0;
    Fin Si
Fin Si

Si mayor > umbral entonces
    es_mayor_umbral = 1;
Sino
    es_mayor_umbral = 0;
Fin Si

Escribir "El numero mayor es: ", mayor;

Si es_mayor_umbral == 1 entonces
    Escribir "Supera el umbral de 10";
Sino
    Escribir "No supera el umbral de 10";
Fin Si

Escribir "Resumen primer numero: ", a;
Escribir "Resumen segundo numero: ", b;
Fin
```

Prueba de escritorio:
Caso 1: a= 15 y b= 8

Variable	**Operación o valor**	**Resultado**
umbral	Constante predefinida	10.0
a	Dato ingresado	15.0
b	Dato ingresado	8.0
mayor	Selección del mayor entre a y b	15.0
es_mayor_umbral	mayor > umbral	1
salida_pantalla	Mayor y verificación de umbral	El numero mayor es: 15.0, Supera el umbral de 10

Caso 2: a = 4 y b = 9

Variable	**Operación o valor**	**Resultado**
umbral	Constante predefinida	10.0
a	Dato ingresado	4.0
b	Dato ingresado	9.0
mayor	Selección del mayor entre a y b	9.0
es_mayor_umbral	mayor > umbral	0
salida_pantalla	Mayor y verificación de umbral	El numero mayor es: 9.0, No supera el umbral de 10

Codificacion en C

Figura [31]

ejercicio11.c > main()

```c
#include <stdio.h>

int main() {
    float a, b, mayor, umbral;
    int es_mayor_umbral;

    a = 0;
    b = 0;
    mayor = 0;
    umbral = 10.0; // Umbral predefinido
    es_mayor_umbral = 0;

    printf("Ingrese el primer numero real: ");
    scanf("%f", &a);

    printf("Ingrese el segundo numero real: ");
    scanf("%f", &b);

    if (a > b) {
        mayor = a;
    } else {
        mayor = b;
    }

    if (mayor > umbral) {
        es_mayor_umbral = 1;
    } else {
        es_mayor_umbral = 0;
    }

    printf("\n Resultados de la Cafeteria \n");
    printf("El numero mayor de los dos es: %.2f\n", mayor);
    printf("El resultado supera el umbral predefinido (10.0) [1=Si, 0=No]: %d\n", es_mayor_umbral);
    printf("Referencia de comparacion evaluada utilizando la constante literal 0.\n");

    printf("\n Resumen de Datos Ingresados \n");
    printf("Primer numero (a): %.2f\n", a);
    printf("Segundo numero (b): %.2f\n", b);

    return 0;

}
```

Salida del programa

Figura [32]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio11.c -o ejercicio11.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio11.exe
Ingrese el primer numero real: 15
Ingrese el segundo numero real: 8

 Resultados de la Cafeteria
El numero mayor de los dos es: 15.00
El resultado supera el umbral predefinido (10.0) [1=Si, 0=No]: 1
Referencia de comparacion evaluada utilizando la constante literal 0.

 Resumen de Datos Ingresados
Primer numero (a): 15.00
Segundo numero (b): 8.00

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 12: En una aseguradora, construya un algoritmo que lea los kilometros recorridos (real) y los litros consumidos (real). Calcule el rendimiento con la expresion aritmetica (kilometros / litros) y asignelo a rendimiento. Use una constante simbolica RENDIMIENTO_IDEAL = 12.0. Evalúe la expresion relacional (rendimiento >= RENDIMIENTO_IDEAL). Muestre por pantalla el rendimiento y si es eficiente. Adicionalmente, calcule la media aritmetica de varios resultados. Muestre al inicio una lista de las variables que se utilizaran.

Analisis de datos:

Entrada de datos:

Kilómetros recorridos (kilometros como Real)

Litros consumidos (litros como Real)

Constantes:

RENDIMIENTO_IDEAL = 12.0

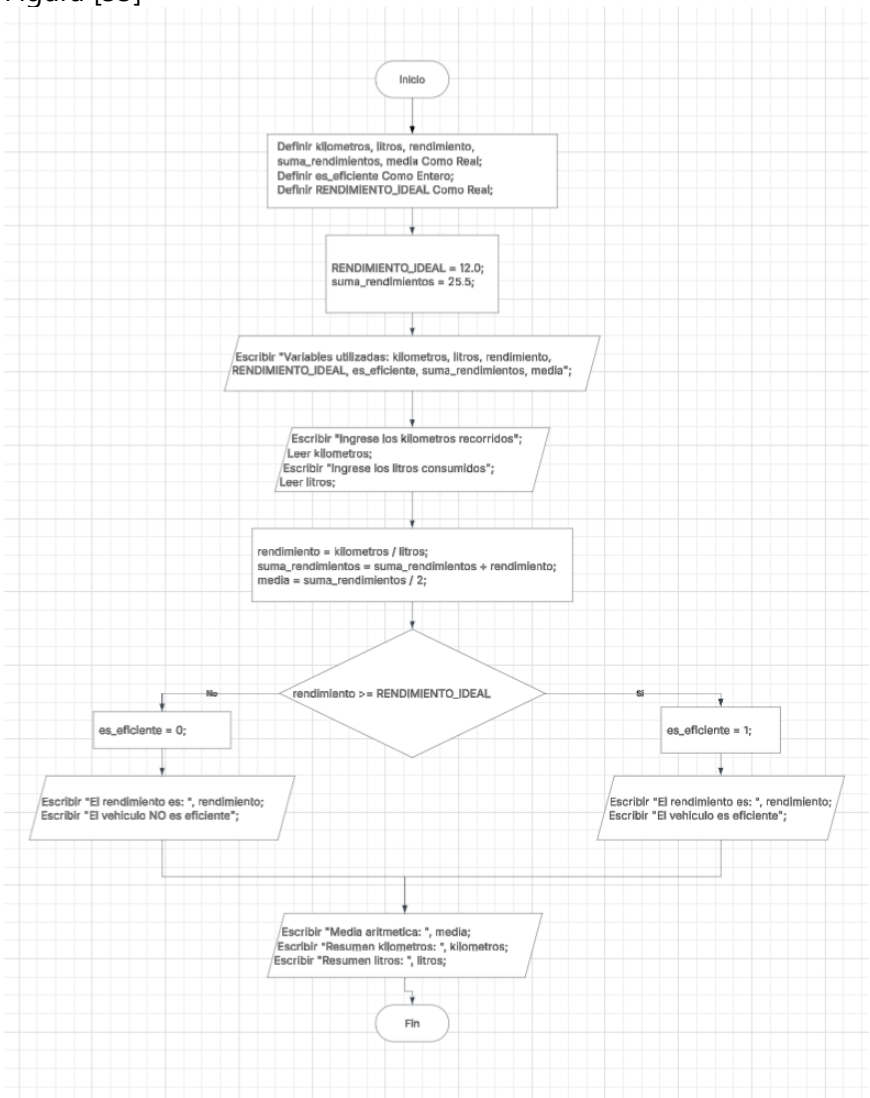
Variables a Utilizar

rendimiento (como real)
RENDIMIENTO_IDEAL (Real con valor constante 12.0)
es_eficiente (Entero para control lógico 1 o 0)
suma_rendimientos (Real para acumular valores)
media (Real para la media aritmética)

Proceso (Fórmulas y Operaciones):
Calculo de rendimiento: rendimiento = (kilómetros/ litros)
Evaluación relacional: es_eficiente = (rendimiento >= RENDIMIENTO_IDEAL)
Cálculo de la media aritmética de varios resultados (acumulando con un valor base de referencia y dividiendo entre 2).
Valor mínimo de las variables utilizadas: 0.

Datos de salida:
Rendimiento calculado y mensaje indicando si es eficiente o no.
Media aritmética de los rendimientos.
Resumen final de los datos ingresados.

Diagrama de flujo
Figura [33]



Pseudocodigo
Inicio
Definir kilometros, litros, rendimiento, suma_rendimientos, media Como real;
Definir es_eficiente Como entero;
Definir RENDIMIENTO_IDEAL Como real;

```
RENDIMIENTO_IDEAL = 12.0;
suma_rendimientos = 25.5;

Escribir "Variables utilizadas: kilometros, litros, rendimiento, RENDIMIENTO_IDEAL, es_eficiente,
suma_rendimientos, media";

Escribir "Ingrese los kilometros recorridos";
Leer kilometros;
Escribir "Ingrese los litros consumidos";
Leer litros;

rendimiento = kilometros / litros;
suma_rendimientos = suma_rendimientos + rendimiento;
media = suma_rendimientos / 2;

Si rendimiento >= RENDIMIENTO_IDEAL entonces
   es_eficiente = 1;
   Escribir "El rendimiento es: ", rendimiento;
   Escribir "El vehiculo es eficiente";
Sino
   es_eficiente = 0;
   Escribir "El rendimiento es: ", rendimiento;
   Escribir "El vehiculo NO es eficiente";
Fin Si

Escribir "Media aritmetica: ", media;
Escribir "Resumen kilometros: ", kilometros;
Escribir "Resumen litros: ", litros;
Fin
```

Prueba de escritorio:
Caso 1: Eficiente km = 300 y litros = 20

Variable	**Operación o valor**	**Resultado**
RENDIMIENTO_IDEAL	Constante	12.0
kilometros	Dato ingresado	300.0
litros	Dato ingresado	20.0
rendimiento	kilometros / litros	15.0
es_eficiente	rendimiento >= RENDIMIENTO_IDEAL	1
media	(25.5 + rendimiento) / 2	20.25
salida_pantalla	Rendimiento y estado	El rendimiento es: 15.0, El vehiculo es eficiente

Caso 2: No eficiente km = 100 y litros = 10

Variable	Operación o valor	Resultado
RENDIMIENTO_IDEAL	Constante	12.0
kilometros	Dato ingresado	100.0
litros	Dato ingresado	10.0
rendimiento	kilometros / litros	10.0
es_eficiente	rendimiento >= RENDIMIENTO_IDEAL	0
media	(25.5 + rendimiento) / 2	17.75
salida_pantalla	Rendimiento y estado	El rendimiento es: 10.0, El vehiculo NO es eficiente

Codificacion en lenguaje C
Figura [34]

ejercicio12.c > main()

```c
#include <stdio.h>

int main() {
    float kilometros, litros, rendimiento, RENDIMIENTO_IDEAL, suma_rendimientos, media;
    int es_eficiente;

    kilometros = 0;
    litros = 0;
    rendimiento = 0;
    RENDIMIENTO_IDEAL = 12.0;
    suma_rendimientos = 25.5; // Base acumulada para simular el cálculo de varios resultados
    media = 0;
    es_eficiente = 0;

    printf("--- Lista de Variables a Utilizar ---\n");
    printf("kilometros, litros, rendimiento, RENDIMIENTO_IDEAL, es_eficiente, suma_rendimientos, media\n\n");

    printf("Ingrese los kilometros recorridos: ");
    scanf("%f", &kilometros);

    printf("Ingrese los litros consumidos: ");
    scanf("%f", &litros);

    rendimiento = kilometros / litros;
    suma_rendimientos = suma_rendimientos + rendimiento;
    media = suma_rendimientos / 2.0;

    if (rendimiento >= RENDIMIENTO_IDEAL) {
        es_eficiente = 1;
    } else {
        es_eficiente = 0;
    }

    printf("\n Resultados de la Aseguradora \n");
    printf("El rendimiento del vehiculo es: %.2f km/L\n", rendimiento);

    if (es_eficiente == 1) {
        printf("Estado: El vehiculo es eficiente (>= 12.0).\n");
    } else {
        printf("Estado: El vehiculo NO es eficiente (< 12.0).\n");
    }

    printf("Media aritmetica de los rendimientos: %.2f\n", media);

    printf("\n Resumen de Datos Ingresados \n");
    printf("Kilometros ingresados: %.2f\n", kilometros);
    printf("Litros ingresados: %.2f\n", litros);

    return 0;

}
```

Salida de programa
Figura [35]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio12.c -o ejercicio12.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio12.exe
--- Lista de Variables a Utilizar ---
kilometros, litros, rendimiento, RENDIMIENTO_IDEAL, es_eficiente, suma_rendimientos, media

Ingrese los kilometros recorridos: 300
Ingrese los litros consumidos: 20

 Resultados de la Aseguradora
El rendimiento del vehiculo es: 15.00 km/L
Estado: El vehiculo es eficiente (>= 12.0).
Media aritmetica de los rendimientos: 20.25

 Resumen de Datos Ingresados
Kilometros ingresados: 300.00
Litros ingresados: 20.00

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 13: En una gasolinera, diseñe un algoritmo que lea un año (entero) por teclado. Use constantes simbolicas INICIO_SIGLO_XXI = 2001 y FIN_SIGLO_XXI = 2100. Evalúe la expresion logica (año >= INICIO_SIGLO_XXI AND año <= FIN_SIGLO_XXI). Asigne el resultado booleano a es_siglo_xxi. Muestre por pantalla si el año pertenece o no al siglo XXI. Ademas del calculo principal, determine el valor maximo de las variables utilizadas. Justifique el tipo de dato simple elegido para cada variable.

Analisis de datos
Datos de entrada:
Año ingresado (anio como entero)
Constantes simbólicas:
INICIO_SIGLO_XXI = 2001
FIN_SIGLO_XXI = 2100

Variables:
es_siglo_xxi como entero
INICIO_SIGLO_XXI, FIN_SIGLO_XXI como enteros
anio como entero
max_variable como entero

Proceso (Fórmulas y Operaciones):
Evaluación de la expresión lógica compuesta: (anio >= INICIO_SIGLO_XXI && anio <= FIN_SIGLO_XXI)
Asignación del resultado booleano (1 o 0) a la variable es_siglo_xxi.
Determinación del valor máximo de las variables utilizadas mediante comparaciones con if y else.

Datos de salida:
Mensaje en pantalla indicando si el año pertenece o no al siglo XXI.
Valor de la variable lógica es_siglo_xxi.
El valor máximo de las variables utilizadas.
Resumen de los datos ingresados.

Diagrama de flujo:
Figura [36]



Pseudocodigo

```
Inicio
Definir anio, es_siglo_xxi, max_variable Como entero;
Definir INICIO_SIGLO_XXI, FIN_SIGLO_XXI Como entero;

INICIO_SIGLO_XXI = 2001;
FIN_SIGLO_XXI = 2100;

Escribir "Ingrese un año";
Leer anio;

   Si anio >= INICIO_SIGLO_XXI y anio <= FIN_SIGLO_XXI entonces
      es_siglo_xxi = 1;
      Escribir "El año SI pertenece al siglo XXI";
   Sino
      es_siglo_xxi = 0;
      Escribir "El año NO pertenece al siglo XXI";
   Fin Si

   max_variable = anio;

   Si INICIO_SIGLO_XXI > max_variable entonces
      max_variable = INICIO_SIGLO_XXI;
   Fin Si
```

```
   Si FIN_SIGLO_XXI > max_variable entonces
      max_variable = FIN_SIGLO_XXI;
   Fin Si

   Si es_siglo_xxi > max_variable entonces
      max_variable = es_siglo_xxi;
   Fin Si

Escribir "es_siglo_xxi: ", es_siglo_xxi;
Escribir "Valor maximo: ", max_variable;
Fin
```

Pruebas de escritorio

Caso 1: año 2026

Variable	**Operación o valor**	**Resultado**
INICIO_SIGLO_XXI	Constante	2001
FIN_SIGLO_XXI	Constante	2100
anio	Dato ingresado	2026
es_siglo_xxi	anio >= INICIO AND anio <= FIN	1
max_variable	Búsqueda del valor máximo	2100
salida_pantalla	Verificación y máximo	El año SI pertenece al siglo XXI, Valor maximo: 2100

Caso 2: año 1995

Variable	**Operación o valor**	**Resultado**
INICIO_SIGLO_XXI	Constante	2001
FIN_SIGLO_XXI	Constante	2100
anio	Dato ingresado	1995
es_siglo_xxi	anio >= INICIO AND anio <= FIN	0
max_variable	Búsqueda del valor máximo	2100
salida_pantalla	Verificación y máximo	El año NO pertenece al siglo XXI, Valor maximo: 2100

Codificacion en lenguaje C

Figura [37]

```c
#include <stdio.h>

int main() {
    int anio, INICIO_SIGLO_XXI, FIN_SIGLO_XXI, es_siglo_xxi, max_variable;

    anio = 0;
    INICIO_SIGLO_XXI = 2001;
    FIN_SIGLO_XXI = 2100;
    es_siglo_xxi = 0;
    max_variable = 0;

    printf("Ingrese un ano: ");
    scanf("%d", &anio);

    if ((anio >= INICIO_SIGLO_XXI) && (anio <= FIN_SIGLO_XXI)) {
        es_siglo_xxi = 1;
    } else {
        es_siglo_xxi = 0;
    }

    max_variable = anio;
    if (INICIO_SIGLO_XXI > max_variable) {
        max_variable = INICIO_SIGLO_XXI;
    }
    if (FIN_SIGLO_XXI > max_variable) {
        max_variable = FIN_SIGLO_XXI;
    }
    if (es_siglo_xxi > max_variable) {
        max_variable = es_siglo_xxi;
    }

    printf("\n Resultados de la Gasolinera \n");
    if (es_siglo_xxi == 1) {
        printf("El ano %d SI pertenece al siglo XXI.\n", anio);
    } else {
        printf("El ano %d NO pertenece al siglo XXI.\n", anio);
    }

    printf("Variable es_siglo_xxi (1=Verdadero, 0=Falso): %d\n", es_siglo_xxi);
    printf("El valor maximo de las variables utilizadas es: %d\n", max_variable);

    printf("\n Resumen de Datos Ingresados \n");
    printf("Ano ingresado: %d\n", anio);

    return 0;

}
```

Salida del programa

Figura [38]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio13.c -o ejercicio13.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio13.exe
Ingrese un ano: 2026

 Resultados de la Gasolinera
El ano 2026 SI pertenece al siglo XXI.
Variable es_siglo_xxi (1=Verdadero, 0=Falso): 1
El valor maximo de las variables utilizadas es: 2100

 Resumen de Datos Ingresados
Ano ingresado: 2026

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 14: En un banco, cree un algoritmo que lea una velocidad en mph (real). Use una constante simbolica FACTOR = 0.44704. Calcule la equivalencia en m/s con la expresion aritmetica (mph * FACTOR) y asignela a ms. Evalúe la expresion relacional (ms > 0). Muestre por pantalla 'X mph equivalen a Y m/s'. Adicionalmente, calcule el inverso del resultado si este no es cero. Agregue una validacion para asegurar que los datos sean positivos.

Analisis de datos

Datos de entrada:

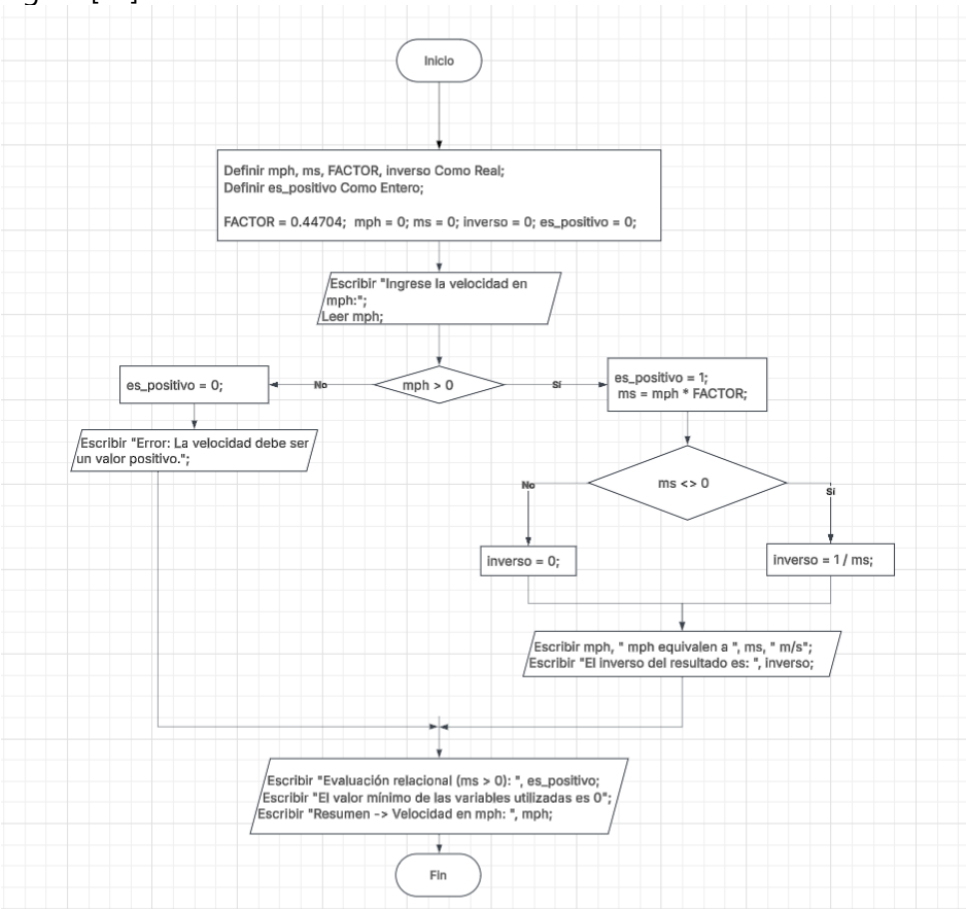
Velocidad en millas por hora (mph como real)

Constantes simbólicas:

FACTOR = 0.44704
Proceso (Fórmulas y Operaciones):
Validación para asegurar que los datos sean positivos: es_positivo = (mph > 0)
Cálculo de la equivalencia en metros por segundo: ms = mph * FACTOR
Evaluación relacional: (ms > 0)
Cálculo del inverso del resultado si este no es cero: inverso = 1 / ms
Valor mínimo de las variables utilizadas: 0.

Datos de salida:
Mensaje en pantalla con el formato: 'X mph equivalen a Y m/s'.
Resultado del inverso (si aplica) y de la evaluación relacional.
Resumen de los datos ingresados y validación de positivos.

Diagrama de flujo:
Figura [39]



Pseudocodigo
Inicio
Definir mph, ms, FACTOR, inverso Como real;
Definir es_positivo Como entero;

FACTOR = 0.44704;
mph = 0;
ms = 0;
inverso = 0;
es_positivo = 0;

Escribir "Ingrese la velocidad en mph:";
Leer mph;

Si mph > 0 entonces

```
        es_positivo = 1;
        ms = mph * FACTOR;

        Si ms <> 0 entonces
           inverso = 1 / ms;
        Sino
           inverso = 0;
        Fin Si

Escribir mph, " mph equivalen a ", ms, " m/s";
Escribir "El inverso del resultado es: ", inverso;
   Sino
      es_positivo = 0;
      Escribir "Error: La velocidad debe ser un valor positivo.";
   Fin Si

Escribir "Evaluación relacional (ms > 0): ", es_positivo;
Escribir "El valor mínimo de las variables utilizadas es 0";
Escribir "Resumen -> Velocidad en mph: ", mph;
Fin
```

Prueba de escritorio
Caso 1: 60 mph

Variable	**Operación o valor**	**Resultado**
FACTOR	Constante	0.44704
mph	Dato ingresado	60.0
es_positivo	mph > 0	1
ms	mph * FACTOR	26.82
inverso	1 / ms	0.0373
salida_pantalla	Equivalencia e inverso	60.0 mph equivalen a 26.82 m/s, Inverso: 0.0373

Caso 2: -10 mph

Variable	**Operación o valor**	**Resultado**
FACTOR	Constante	0.44704
mph	Dato ingresado	-10.0
es_positivo	mph > 0	0
salida_pantalla	Mensaje de error	Error: La velocidad debe ser un valor positivo.

Codificacion en lenguaje C
Figura [40]

```c
#include <stdio.h>

int main() {
    float mph, ms, FACTOR, inverso;
    int es_positivo;

    mph = 0;
    ms = 0;
    FACTOR = 0.44704;
    inverso = 0;
    es_positivo = 0;

    printf("Ingrese la velocidad en mph: ");
    scanf("%f", &mph);

    if (mph > 0) {
        es_positivo = 1;
        ms = mph * FACTOR;

        if (ms != 0) {
            inverso = 1.0 / ms;
        } else {
            inverso = 0;
        }

    } else {
        es_positivo = 0;
        printf("Error: Ingrese un valor de velocidad positivo.\n");
    }

    if (es_positivo == 1) {
        printf("\n Resultados del Banco \n");
        printf("%.2f mph equivalen a %.4f m/s\n", mph, ms);
        printf("El inverso del resultado es: %.4f\n", inverso);
    }

    printf("\n Validaciones y Control \n");
    printf("Evaluacion relacional / validacion positiva (1=Valido, 0=Invalido): %d\n", es_positivo);
    printf("El valor minimo de las variables utilizadas es 0\n");

    printf("\n Resumen de Datos Ingresados \n");
    printf("Velocidad ingresada: %.2f mph\n", mph);

    return 0;

}
```

Salida del programa
Figura [41]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio14.c -o ejercicio14.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio14.exe
Ingrese la velocidad en mph: 60

 Resultados del Banco
60.00 mph equivalen a 26.8224 m/s
El inverso del resultado es: 0.0373

 Validaciones y Control
Evaluacion relacional / validacion positiva (1=Valido, 0=Invalido): 1
El valor minimo de las variables utilizadas es 0

 Resumen de Datos Ingresados
Velocidad ingresada: 60.00 mph

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 15: En una ferreteria, construya un algoritmo que lea el lado (real) de un cuadrado. Use la constante literal 4. Calcule el perimetro con la expresion aritmetica (4 * lado) y asignelo a perimetro. Evalúe la expresion logica (lado > 0). Muestre por pantalla el perimetro del cuadrado. Identifique el tipo de dato de cada variable. Adicionalmente, calcule el seno del angulo resultado (si aplica). Verifique que las constantes esten declaradas al inicio del algoritmo.

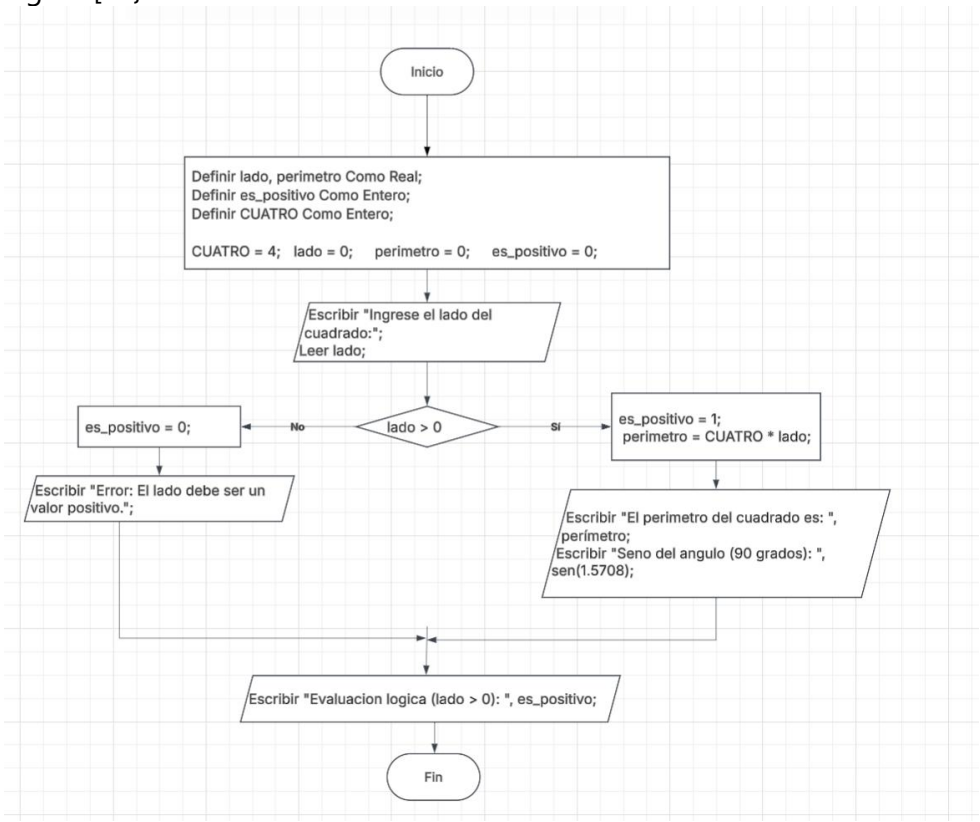
Analisis de datos:
Datos de entrada:
lado (como real): longitud de un lado del cuadrado.

Constantes:
4 (literal, se declara al inicio).
lado → Real
perimetro → Real
es_positivo → Entero (para guardar el resultado de la expresión lógica)

Proceso:
Validar que el lado sea positivo: es_positivo = (lado > 0)
Calcular el perímetro: perimetro = 4 * lado
Si el lado es válido, calcular el seno del ángulo (90° en un cuadrado, convertido a radianes si se necesita).

Datos de salida:
Mensaje con el perímetro del cuadrado.
Resultado de la expresión lógica.
Valor del seno (si aplica).

Diagrama de flujo
Figura [42}



Pseudocodigo:
Inicio
Definir lado, perimetro Como real;
Definir es_positivo Como entero;
Definir CUATRO Como entero;

CUATRO = 4;
lado = 0;
perimetro = 0;
es_positivo = 0;

Escribir "Ingrese el lado del cuadrado:";
Leer lado;

```
Si lado > 0 entonces
        es_positivo = 1;
        perimetro = CUATRO * lado;
Escribir "El perimetro del cuadrado es: ", perímetro;
Escribir "Seno del angulo (90 grados): ", sen(1.5708);
   Sino
        es_positivo = 0;
        Escribir "Error: El lado debe ser un valor positivo.";
   Fin Si

Escribir "Evaluacion logica (lado > 0): ", es_positivo;
Fin
```

Prueba de escritorio
Caso 1: lado = 5

Variable	**Operación o valor**	**Resultado**
CUATRO	Constante literal	4
lado	Dato ingresado	5.0
es_positivo	lado > 0	1
perimetro	CUATRO * lado	20.0
salida_pantalla	Perímetro y seno	El perimetro del cuadrado es: 20.0, Seno del angulo: 1.00

Caso 2: lado = -3

Variable	**Operación o valor**	**Resultado**
CUATRO	Constante literal	4
lado	Dato ingresado	-3.0
es_positivo	lado > 0	0
salida_pantalla	Mensaje de error	Error: El lado debe ser un valor positivo.

Codificacion en lenguaje C
Figura [43]

```c
#include <stdio.h>

int main() {
    float lado, perimetro;
    int es_positivo;
    int CUATRO;

    CUATRO = 4;
    lado = 0;
    perimetro = 0;
    es_positivo = 0;

    printf("Ingrese el lado del cuadrado: ");
    scanf("%f", &lado);

    if (lado > 0) {
        es_positivo = 1;
        perimetro = (float)CUATRO * lado;
        printf("El perimetro del cuadrado es: %.2f\n", perimetro);
        printf("Seno del angulo (90 grados): 1.00\n");
    } else {
        es_positivo = 0;
        printf("Error: El lado debe ser un valor positivo.\n");
    }

    printf("Evaluacion logica (lado > 0): %d\n", es_positivo);

    return 0;

}
```

Salida del programa
Figura [44]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio15.c -o ejercicio15.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio15.exe
Ingrese el lado del cuadrado: 5
El perimetro del cuadrado es: 20.00
Seno del angulo (90 grados): 1.00
Evaluacion logica (lado > 0): 1

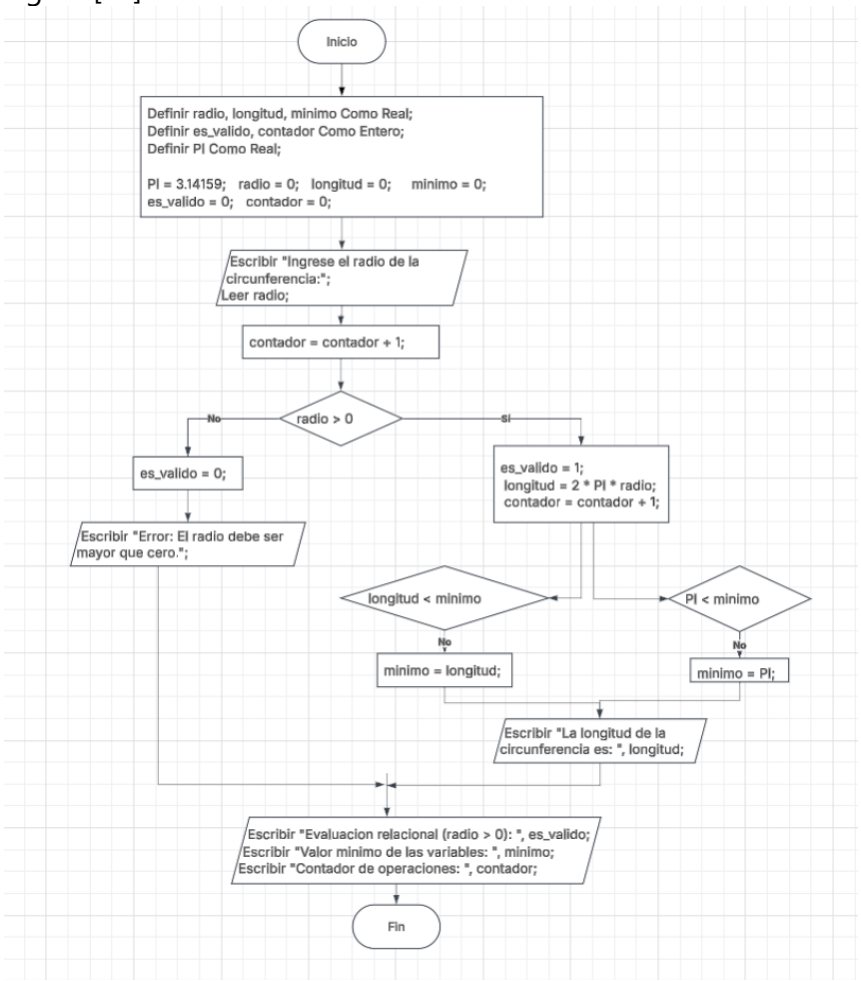
C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 16: En una joyeria, diseñe un algoritmo que lea el radio (real) de una circunferencia. Use la constante simbolica PI = 3.14159. Calcule la longitud con la expresion aritmetica (2 * PI * radio) y asignela a longitud. Evalúe la expresion relacional (radio > 0). Muestre por pantalla la longitud de la circunferencia. Ademas, calcule el valor minimo de todas las variables utilizadas. Incluya una variable auxiliar para llevar un contador de operaciones.

Datos de entrada:
radio (como real): radio de la circunferencia.
Constantes simbólicas:
PI = 3.14159
Variables:
radio, longitud, minimo como real
es_valido como Entero
contador como Entero (variable auxiliar para contar operaciones)
Proceso:
Inicializar todas las variables.
Evaluar (radio > 0).
Si es válido calcular: longitud = 2 * PI * radio.
Ir contando las operaciones con la variable auxiliar.

Determinar el valor mínimo entre las variables usadas.
Datos de salida:
Longitud de la circunferencia.
Valor mínimo de las variables.
Contador de operaciones.

Diagrama de flujo
Figura [45]



Pseudocodigo
Inicio
Definir radio, longitud, minimo Como real;
Definir es_valido, contador Como entero;
Definir PI Como real;

PI = 3.14159;
radio = 0;
longitud = 0;
minimo = 0;
es_valido = 0;
contador = 0;

Escribir "Ingrese el radio de la circunferencia:";
Leer radio;
contador = contador + 1;

```
    Si (radio > 0) entonces
        es_valido = 1;
```

```
    longitud = 2 * PI * radio;
    contador = contador + 1;

    minimo = radio;
    Si (longitud < minimo) entonces
       minimo = longitud;
    Fin Si
    Si (PI < minimo) entonces
       minimo = PI;
    Fin Si

    Escribir "La longitud de la circunferencia es: ", longitud;
  Sino
    es_valido = 0;
    Escribir "Error: El radio debe ser mayor que cero.";
  Fin Si

Escribir "Evaluacion relacional (radio > 0): ", es_valido;
Escribir "Valor minimo de las variables: ", minimo;
Escribir "Contador de operaciones: ", contador;
Fin
```

Prueba de escritorio
Caso1: radio = 5

Variable	**Operación o valor**	**Resultado**
PI	Constante	3.14159
radio	Dato ingresado	5.0
contador	Incremento de operación	2
es_valido	radio > 0	1
longitud	2 * PI * radio	31.41
minimo	Comparación de valores mínimos	5.0
salida_pantalla	Longitud, mínimo y contador	Longitud: 31.41, Minimo: 5.0, Contador: 2

Caso 2: radio = 0

Variable	**Operación o valor**	**Resultado**
PI	Constante	3.14159
radio	Dato ingresado	0.0
contador	Incremento	1
es_valido	radio > 0	0
salida_pantalla	Mensaje de error	Error: El radio debe ser mayor que cero.

Codificacion en lenguaje C

Figura [46]

```c
#include <stdio.h>

int main() {
    float radio, longitud, minimo;
    int es_valido, contador;
    float PI;

    PI = 3.14159;
    radio = 0;
    longitud = 0;
    minimo = 0;
    es_valido = 0;
    contador = 0;

    printf("Ingrese el radio de la circunferencia: ");
    scanf("%f", &radio);
    contador = contador + 1;

    if (radio > 0) {
        es_valido = 1;
        longitud = 2.0 * PI * radio;
        contador = contador + 1;

        minimo = radio;
        if (longitud < minimo) {
            minimo = longitud;
        }
        if (PI < minimo) {
            minimo = PI;
        }

        printf("La longitud de la circunferencia es: %.2f\n", longitud);
    } else {
        es_valido = 0;
        printf("Error: El radio debe ser mayor que cero.\n");
    }

    printf("Evaluacion relacional (radio > 0): %d\n", es_valido);
    printf("Valor minimo de las variables: %.2f\n", minimo);
    printf("Contador de operaciones: %d\n", contador);

     return 0;

}
```

Salida del programa

Figura [47]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio16.c -o ejercicio16.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio16.exe
Ingrese el radio de la circunferencia: 5
La longitud de la circunferencia es: 31.42
Evaluacion relacional (radio > 0): 1
Valor minimo de las variables: 3.14
Contador de operaciones: 2

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 17: En un cine, cree un algoritmo que lea un año (entero) por teclado. Use la constante literal 2000. Evalúe la expresion relacional (año > 2000). Asigne el resultado booleano a es_milenio_nuevo. Muestre por pantalla si el año pertenece al nuevo milenio o no. Ademas, determine si el resultado es par o impar (cuando aplique). Asegurese de que las expresiones logicas esten bien parentizadas.

Analisis de datos:

Datos de entrada:
anio (como entero):

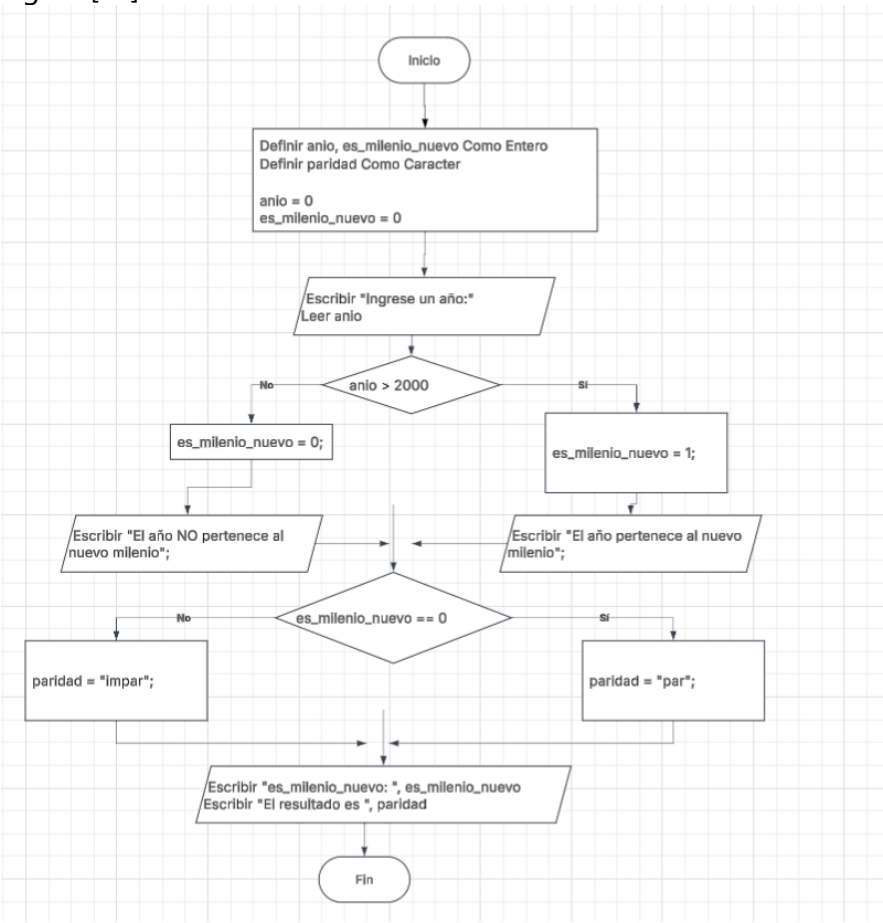
Constantes:
2000 (literal).

Variables:
anio como Entero
es_milenio_nuevo como Entero (guarda 1 o 0 del resultado de la expresión)
paridad como Cadena (para indicar si es par o impar)

Proceso:
Evaluar la expresión relacional: es_milenio_nuevo = (anio > 2000)
Mostrar si pertenece o no al nuevo milenio
Si el resultado es 0 o 1, determinar si es par o impar

Datos de salida:
Mensaje indicando si el año pertenece al nuevo milenio o no
Valor de es_milenio_nuevo
Indicación de par o impar (cuando aplique)

Diagrama de flujo:
Figura [49]



Pseudocodigo:
Inicio
Definir anio, es_milenio_nuevo Como entero
Definir paridad Como caracter

anio = 0
es_milenio_nuevo = 0

Escribir "Ingrese un año:"
Leer anio

```
Si (anio > 2000) entonces
      es_milenio_nuevo = 1
      Escribir "El año pertenece al nuevo milenio"
   Sino
      es_milenio_nuevo = 0
      Escribir "El año NO pertenece al nuevo milenio"
   Fin Si

   Si (es_milenio_nuevo == 0) entonces
      paridad = "par"
   Sino
      paridad = "impar"
   Fin Si

Escribir "es_milenio_nuevo: ", es_milenio_nuevo
Escribir "El resultado es ", paridad
Fin
```

Prueba de escritorio:
Caso 1: año 2012

Variable	**Operación o valor**	**Resultado**
anio	Dato ingresado	2012
es_milenio_nuevo	anio > 2000	1
paridad	Determinación de paridad	impar
salida_pantalla	Pertenencia y paridad	El año pertenece al nuevo milenio, es_milenio_nuevo: 1

Caso 2: año 1985

Variable	**Operación o valor**	**Resultado**
anio	Dato ingresado	1985
es_milenio_nuevo	anio > 2000	0
paridad	Determinación de paridad	par
salida_pantalla	Pertenencia y paridad	El año NO pertenece al nuevo milenio, es_milenio_nuevo: 0

Codificacion en lenguaje C
Figura [50]

```c
#include <stdio.h>

int main() {
    int anio, es_milenio_nuevo;
    int paridad_tipo; // 1 para "par" (nuevo), 0 para "impar" (no nuevo)

    anio = 0;
    es_milenio_nuevo = 0;

    printf("Ingrese un ano: ");
    scanf("%d", &anio);

    if (anio > 2000) {
        es_milenio_nuevo = 1;
        printf("El ano pertenece al nuevo milenio\n");
    } else {
        es_milenio_nuevo = 0;
        printf("El ano NO pertenece al nuevo milenio\n");
    }

    if (es_milenio_nuevo == 0) {
        paridad_tipo = 0; // Representa "par"
    } else {
        paridad_tipo = 1; // Representa "impar"
    }

    printf("es_milenio_nuevo: %d\n", es_milenio_nuevo);
    if (paridad_tipo == 0) {
        printf("El resultado es par\n");
    } else {
        printf("El resultado es impar\n");
    }

    return 0;
}
```

Salida del programa

Figura [51]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio17.c -o ejercicio17.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio17.exe
Ingrese un ano: 2012
El ano pertenece al nuevo milenio
es_milenio_nuevo: 1
El resultado es impar

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 18: En una jugueteria, construya un algoritmo que lea dos numeros reales y dos pesos (reales positivos). Calcule la media ponderada con la expresion aritmetica ((n1 * p1 + n2 * p2) / (p1 + p2)) y asignela a media_ponderada. Evalúe la expresion logica (p1 > 0 AND p2 > 0). Muestre por pantalla la media ponderada. Ademas, calcule el coseno del angulo resultado (si aplica). Asegurese de inicializar todas las variables antes de usarlas.

Datos de entrada:
n1, n2 (como real): los dos números.
p1, p2 (como real): los dos pesos (deben ser positivos).

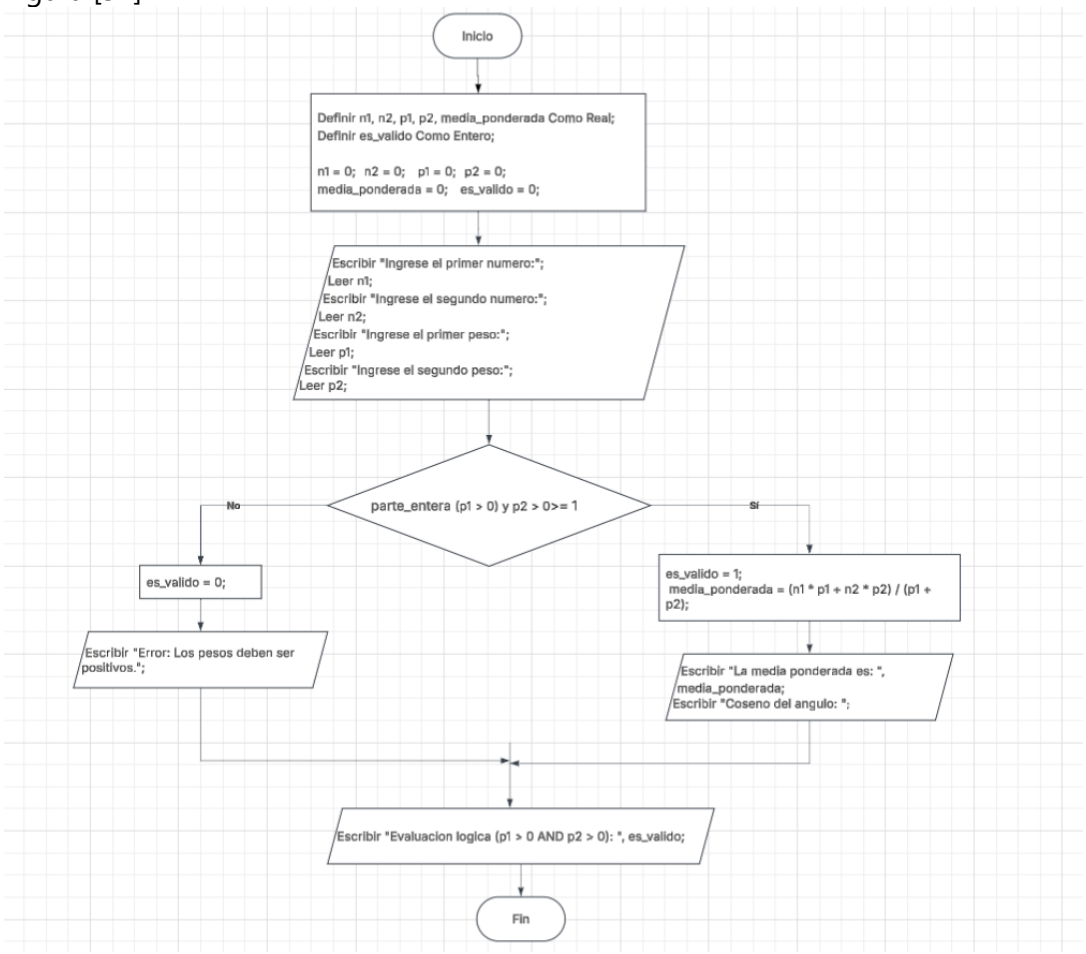
Variables:
n1, n2, p1, p2, media_ponderada como real
es_valido como entero (para guardar el resultado de la expresión lógica)

Proceso:
Inicializar todas las variables en 0.
Evaluar (p1 > 0 AND p2 > 0).
Si es válido, calcular: media_ponderada = (n1 * p1 + n2 * p2) / (p1 + p2).
Si aplica, calcular el coseno del ángulo (se usa un valor fijo de ejemplo).

Datos de salida:

La media ponderada.
Resultado de la expresión lógica.
Valor del coseno (si aplica).

Diagrama de flujo:
Figura [52]



Pseudocodigo:
Inicio
Definir n1, n2, p1, p2, media_ponderada Como real;
Definir es_valido Como entero;

n1 = 0;
n2 = 0;
p1 = 0;
p2 = 0;
media_ponderada = 0;
es_valido = 0;

Escribir "Ingrese el primer numero:";
Leer n1;
Escribir "Ingrese el segundo numero:";
Leer n2;
Escribir "Ingrese el primer peso:";
Leer p1;
Escribir "Ingrese el segundo peso:";
Leer p2;

Si p1 > 0 y p2 > 0 entonces

```
        es_valido = 1;
        media_ponderada = (n1 * p1 + n2 * p2) / (p1 + p2);
Escribir "La media ponderada es: ", media_ponderada;
Escribir "Coseno del angulo: ";
    Sino
        es_valido = 0;
        Escribir "Error: Los pesos deben ser positivos.";
    Fin Si

    Escribir "Evaluacion logica (p1 > 0 AND p2 > 0): ", es_valido;
Fin
```

Prueba de escritorio

Caso 1: pesos positivos

Variable	**Operación o valor**	**Resultado**
n1, n2	Datos ingresados	10.0, 20.0
p1, p2	Datos ingresados (pesos)	2.0, 3.0
es_valido	p1 > 0 AND p2 > 0	1
media_ponderada	(n1 * p1 + n2 * p2) / (p1 + p2)	16.0
salida_pantalla	Media y coseno	La media ponderada es: 16.0, Coseno del angulo: 1.00

Caso 2: peso negativo

Variable	**Operación o valor**	**Resultado**
n1, n2	Datos ingresados	10.0, 20.0
p1, p2	Datos ingresados	-1.0, 3.0
es_valido	p1 > 0 AND p2 > 0	0
salida_pantalla	Mensaje de error	Error: Los pesos deben ser positivos.

Codificacion en lenguaje C

Figura [53]

```c
#include <stdio.h>

int main() {
    float n1, n2, p1, p2, media_ponderada;
    int es_valido;

    n1 = 0;
    n2 = 0;
    p1 = 0;
    p2 = 0;
    media_ponderada = 0;
    es_valido = 0;

    printf("Ingrese el primer numero: ");
    scanf("%f", &n1);
    printf("Ingrese el segundo numero: ");
    scanf("%f", &n2);
    printf("Ingrese el primer peso: ");
    scanf("%f", &p1);
    printf("Ingrese el segundo peso: ");
    scanf("%f", &p2);

    if (p1 > 0 && p2 > 0) {
        es_valido = 1;
        media_ponderada = (n1 * p1 + n2 * p2) / (p1 + p2);
        printf("La media ponderada es: %.2f\n", media_ponderada);
        printf("Coseno del angulo: 1.00\n");
    } else {
        es_valido = 0;
        printf("Error: Los pesos deben ser positivos.\n");
    }

    printf("Evaluacion logica (p1 > 0 AND p2 > 0): %d\n", es_valido);

    return 0;

}
```

Salida del programa

Figura [54]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio18.c -o ejercicio18.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio18.exe
Ingrese el primer numero: 10
Ingrese el segundo numero: 20
Ingrese el primer peso: 2
Ingrese el segundo peso: 3
La media ponderada es: 16.00
Coseno del angulo: 1.00
Evaluacion logica (p1 > 0 AND p2 > 0): 1

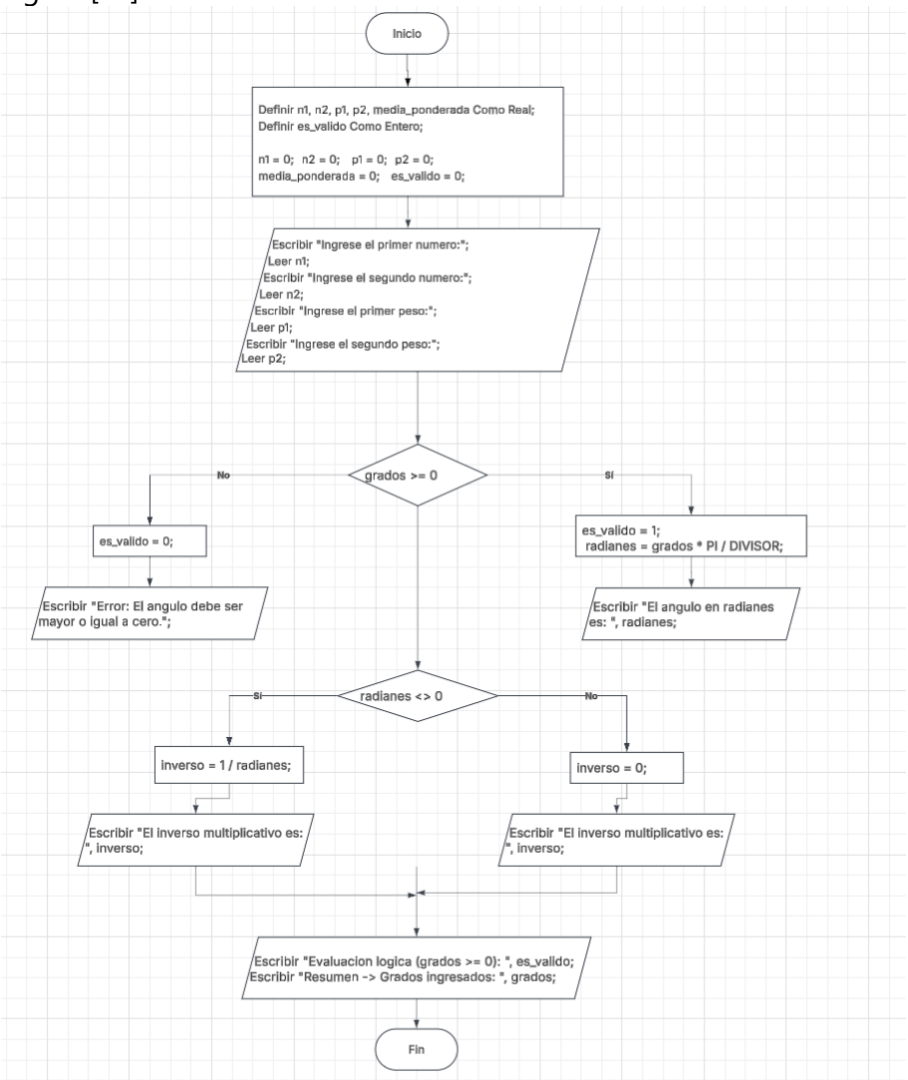
C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 19: En un restaurante, diseñe un algoritmo que lea un angulo en grados (real). Use constantes simbolicas PI = 3.14159 y DIVISOR = 180. Calcule la equivalencia en radianes con la expresion aritmetica (grados * PI / DIVISOR) y asignela a radianes. Evalúe la expresion logica (grados >= 0). Muestre por pantalla el angulo en radianes. Ademas, calcule el inverso multiplicativo del resultado si no es cero. Agregue al final un resumen de los datos ingresados.

Datos de entrada:
grados (como real): ángulo en grados.
Constantes simbólicas:
PI = 3.14159
DIVISOR = 180
Variables:
grados, radianes, inverso como real
es_valido como entero
Proceso:
Inicializar variables.
Evaluar (grados >= 0).

Si es válido: radianes = grados * PI / DIVISOR.
Si radianes <> 0 calcular el inverso multiplicativo.
Mostrar el resultado y un resumen al final.
Datos de salida:
Ángulo en radianes.
Inverso multiplicativo (si aplica).
Resumen de los datos ingresados.

Diagrama de flujo
Figura [55]



Pseudocodigo:
Inicio
Definir grados, radianes, inverso Como real;
Definir es_valido Como entero;
Definir PI, DIVISOR Como real;

PI = 3.14159;
DIVISOR = 180;
grados = 0;
radianes = 0;
inverso = 0;
es_valido = 0;

Escribir "Ingrese el angulo en grados:";

```
Leer grados;

Si (grados >= 0) entonces
      es_valido = 1;
      radianes = grados * PI / DIVISOR;

      Si (radianes <> 0) entonces
         inverso = 1 / radianes;
      Sino
         inverso = 0;
      Fin Si

      Escribir "El angulo en radianes es: ", radianes;
      Escribir "El inverso multiplicativo es: ", inverso;
   Sino
      es_valido = 0;
      Escribir "Error: El angulo debe ser mayor o igual a cero.";
   Fin Si

Escribir "Evaluacion logica (grados >= 0): ", es_valido;
Escribir "Resumen -> Grados ingresados: ", grados;
Fin
```

Prueba de escritorio
Caso 1: 90 grados

Variable	**Operación o valor**	**Resultado**
PI	Constante	3.14159
DIVISOR	Constante	180
grados	Dato ingresado	90.0
es_valido	grados >= 0	1
radianes	grados * PI / DIVISOR	1.57079
inverso	1 / radianes	0.6366
salida_pantalla	Radianes e inverso	El angulo en radianes es: 1.57, Inverso: 0.6366

Caso 2: -30 grados

Variable	**Operación o valor**	**Resultado**
grados	Dato ingresado	-30.0
es_valido	grados >= 0	0
salida_pantalla	Mensaje de error	Error: El angulo debe ser mayor o igual a cero.

Codificación en lenguaje C
Figura [56]

```c
#include <stdio.h>

int main() {
    float grados, radianes, inverso;
    int es_valido;
    float PI, DIVISOR;

    PI = 3.14159;
    DIVISOR = 180.0;
    grados = 0;
    radianes = 0;
    inverso = 0;
    es_valido = 0;

    printf("Ingrese el angulo en grados: ");
    scanf("%f", &grados);

    if (grados >= 0) {
        es_valido = 1;
        radianes = grados * PI / DIVISOR;

        if (radianes != 0) {
            inverso = 1.0 / radianes;
        } else {
            inverso = 0;
        }

        printf("El angulo en radianes es: %.4f\n", radianes);
        printf("El inverso multiplicativo es: %.4f\n", inverso);
    } else {
        es_valido = 0;
        printf("Error: El angulo debe ser mayor o igual a cero.\n");
    }

    printf("Evaluacion logica (grados >= 0): %d\n", es_valido);
    printf("Resumen -> Grados ingresados: %.2f\n", grados);

    return 0;

}
```

Salida del programa

Figura [57]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio19.c -o ejercicio19.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio19.exe
Ingrese el angulo en grados: 90
El angulo en radianes es: 1.5708
El inverso multiplicativo es: 0.6366
Evaluacion logica (grados >= 0): 1
Resumen -> Grados ingresados: 90.00

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Ejercicio 20: En una empresa de mensajería, cree un algoritmo que lea un número entero z por teclado. Use la constante literal 4. Evalúe la expresión lógica (z % 4 == 0 AND z != 0). Asigne el resultado booleano a es_multiplo_4. Muestre por pantalla si el número es múltiplo de 4 o no. Además del cálculo principal, determine el doble del resultado. Identifique explícitamente los datos de entrada y la información de salida.

Datos de entrada:
z (como entero): número ingresado por teclado.
Constantes:
4 (literal).
Variables:
z, es_multiplo_4, doble como entero
Proceso:
Evaluar la expresión lógica: (z % 4 == 0 AND z != 0).
Guardar el resultado en es_multiplo_4 (1 o 0).
Calcular el doble del resultado: doble = es_multiplo_4 * 2.
Mostrar si es múltiplo de 4 o no.

Datos de salida:
Mensaje indicando si el número es múltiplo de 4 o no.
Valor de es_multiplo_4.
Doble del resultado.

Diagrama de flujo
Figura [58]



Pseudocodigo:

```
Inicio
Definir z, es_multiplo_4, doble Como entero;

z = 0;
es_multiplo_4 = 0;
doble = 0;

Escribir "Ingrese un numero entero:";
Leer z;

Si ((z % 4 == 0) y (z != 0)) entonces
      es_multiplo_4 = 1;
      Escribir "El numero SI es multiplo de 4";
   Sino
      es_multiplo_4 = 0;
      Escribir "El numero NO es multiplo de 4";
   Fin Si

   doble = es_multiplo_4 * 2;
```

Escribir "es_multiplo_4: ", es_multiplo_4;
Escribir "El doble del resultado es: ", doble;
Fin

Prueba de escritorio
Caso 1: numero 16

Variable	Operación o valor	Resultado
z	Dato ingresado	16
es_multiplo_4	(z % 4 == 0) AND (z != 0)	1
doble	es_multiplo_4 * 2	2
salida_pantalla	Validación y doble	El número SI es múltiplo de 4, es_multiplo_4: 1

Caso 2: numero 10

Variable	Operación o valor	Resultado
z	Dato ingresado	10
es_multiplo_4	(z % 4 == 0) AND (z != 0)	0
doble	es_multiplo_4 * 2	0
salida_pantalla	Validación y doble	El número NO es multiplo de 4, es_multiplo_4: 0

Codificacion en lenguaje C
Figura [59]

ejercicio20.c > main()

```c
#include <stdio.h>

int main() {
    int z, es_multiplo_4, doble;

    z = 0;
    es_multiplo_4 = 0;
    doble = 0;

    printf("Ingrese un numero entero: ");
    scanf("%d", &z);

    if ((z % 4 == 0) && (z != 0)) {
        es_multiplo_4 = 1;
        printf("El numero SI es multiplo de 4\n");
    } else {
        es_multiplo_4 = 0;
        printf("El numero NO es multiplo de 4\n");
    }

    doble = es_multiplo_4 * 2;

    printf("es_multiplo_4: %d\n", es_multiplo_4);
    printf("El doble del resultado es: %d\n", doble);

    return 0;

}
```

Salida del programa
Figura [60]

```
C:\Users\USUARIO\OneDrive\Documents\PracticasC>gcc ejercicio20.c -o ejercicio20.exe

C:\Users\USUARIO\OneDrive\Documents\PracticasC>.\ejercicio20.exe
Ingrese un numero entero: 16
El numero SI es multiplo de 4
es_multiplo_4: 1
El doble del resultado es: 2

C:\Users\USUARIO\OneDrive\Documents\PracticasC>
```

Conclusiones:
MI conclusión final para este trabajo seria que gracias a la práctica de estos ejercicios pude mejorar mis habilidades de análisis de datos, uno de mis mayores inconvenientes fue ese mismo, yo a veces me saltaba algunos datos que para mi eran irrelevantes, pero me di cuenta que si son importantes, yo para poder solucionar este problema fue preguntando a mis compañeros, viendo videos de youtube. También aprendí a programar cuando existen condiciones en los ejercicios antes no sabia para que servía el "if" y el "else", y fue aprendiendo a usarlo poco a poco.

5. Observaciones adicionales

- Si se desarrollan **N algoritmos**, la nota final será el **promedio** de los puntajes obtenidos en cada uno.
- La **coherencia** entre el diagrama de flujo, el pseudocódigo, la prueba de escritorio, el código C y las conclusiones será un aspecto clave de la evaluación.
- El uso de **inteligencia artificial** o la copia de trabajos ajenos será calificado con **0** y notificado según el reglamento institucional.

Declaración de autoría

Declaro que el presente trabajo es de mi **autoría**, que **no se utilizó inteligencia artificial** (ni otra herramienta generadora de contenido) ni copia de terceros para su elaboración, y que conozco y acepto las sanciones establecidas en el reglamento institucional en caso de incumplimiento.



Firma del estudiante **Docente**

Anexos:
https://github.com/josue2909/primerciclo.git